



Escuela
Politécnica
Superior

Análisis de sentimiento y tendencias en campañas electorales



Máster Universitario en Ciencia de Datos

Trabajo Fin de Máster

Autora:

Jimena Milla Moreno

Tutor:

Miguel Ángel Teruel Martínez

Junio 2026



Universitat d'Alacant
Universidad de Alicante

Análisis de sentimiento y tendencias en campañas electorales

Autora

Jimena Milla Moreno

Tutor

Miguel Ángel Teruel Martínez



Máster Universitario en Ciencia de Datos



Escuela
Politécnica
Superior



Universitat d'Alacant
Universidad de Alicante

ALICANTE, junio 2026

Resumen

Agradecimientos

Índice general

1	Introducción	1
1.1	El sentimiento en Machine Learning	1
1.2	Twitter como fuente de datos	2
1.3	Motivación	3
2	Estado del arte	5
2.1	Tareas de clasificación de opinión en Twitter	5
2.1.1	<i>Sentiment Analysis</i> (Análisis de Sentimiento)	5
2.1.2	<i>Stance Detection</i> (Detección de Postura)	5
2.2	Enfoques en TSA	6
2.3	Preprocesamiento en Twitter	8
2.4	Benchmarks y Evaluación: TweetEval	9
2.4.1	Estructura del benchmark y formato de los datos	9
3	Fundamentos teóricos	15
3.1	Arquitecturas de modelos de lenguaje	15
3.1.1	Transformers	15
3.1.2	BERT: <i>Bidirectional Encoder Representations from Transformers</i>	19
3.1.3	Fine-tuning de modelos preentrenados	24
3.2	Métricas de evaluación	25
3.2.1	Métricas Básicas de Clasificación	26
3.2.2	F1-Score Macro: Métrica Principal	27
3.2.3	F1-Average para <i>Stance Detection</i>	28
4	Metodología	29
4.1	Estrategia de recolección de datos propios	31
4.2	Análisis exploratorio de los datos de TweetEval	33
4.3	Procesamiento de los datos	38
4.3.1	TweetEval	39
4.3.2	Datos Scrapeados	40
4.3.3	Datos de Kaggle	40
4.4	Entrenamiento y modelado	43
4.4.1	Análisis de sentimiento	43
4.4.2	Detección de postura	45
4.5	Inferencia y aplicación de los modelos sobre los datos reales.	46

5	Análisis de resultados	51
5.1	Análisis de sentimiento	51
5.2	Detección de postura	55
5.3	Resultados de la inferencia sobre los datos reales.	57
5.3.1	Análisis de sentimiento.	57
5.3.2	Detección de postura.	57
5.4	Análisis integrado y multipartidismo.	57
6	Conclusiones	59
	Bibliografía	61

Índice de figuras

2.1	Estructura de directorios y archivos del repositorio TweetEval.	10
2.2	Estructura interna de archivos del dataset sentriment para la tarea de <i>Sentiment Analysis</i>	11
2.3	Organización jerárquica del dataset stance para la tarea de <i>Stance Detection</i> por tópicos.	12
3.1	Arquitectura general del transformer. Adaptado de [Pérez, 2020]	16
3.2	Resultado de la enmascaración de la matriz $Q \cdot K$ para una frase de ejemplo. Adaptado de [Pérez, 2020]	18
3.3	Entrenamiento de destilación, para el que no se usa el conjunto de entrenamiento entero sino un conjunto conocido como <i>transfer set</i> . Adaptado de [Mohan, 2022]	22
3.4	Evolución cronológica de los Modelos de Lenguaje basándose en su número de parámetros (en millones). Se observa una tendencia exponencial hacia modelos de gran escala, contrastando con versiones destiladas (DistilBERT) que buscan la eficiencia, lo cual en ocasiones puede ser clave para tareas de análisis en tiempo real, donde la inmediatez es lo primordial. Adaptado de [Pérez, 2020]	24
4.1	Flujo completo de los datos a lo largo de todo el proyecto. Imagen de generación propia.	30
4.2	Distribución de clases por split en el dataset de sentiment.	34
4.3	Distribución de longitud de los tweets en el conjunto de entrenamiento en el dataset de sentimiento.	34
4.4	Tokens más frecuentes en el dataset separando por clase.	35
4.5	<i>Hashtags</i> más frecuentes en el dataset de sentimiento.	35
4.6	Distribuciones porcentuales a favor/neutral/en contra para los distintos tópicos del dataset	36
4.7	Distribuciones del split para los distintos tópicos del dataset de <i>Stance Detection</i>	37
4.8	Longitud de los tweets para los distintos tópicos del dataset stance.	37
4.9	Top 12 tokens más frecuentes para los tweets de los distintos tópicos del dataset de stance.	38

4.10	Contenido de los buckets de Google Cloud Storage tras el procesamiento completo. El bucket <i>raw</i> almacena los datos crudos en formato JSON, organizados por campaña, tarea y target. El bucket <i>processed</i> contiene los datos normalizados en formato Parquet, listos para su uso en los notebooks de modelado.	41
5.1	Distribución de confianza de las clasificaciones correctas frente a las incorrectas y confianza media por clase.	53
5.2	Proyección t-SNE de los embeddings del modelo E0 sobre el test set (n=1284). Perplexity= 30, 1000 iteraciones.	54
5.3	Comparación de las cuatro estrategias (S0–S3) por tópico en términos de F1-Average (izquierda) y F1-Macro (derecha). La línea roja discontinua indica la referencia del paper TweetEval [Barbieri et al., 2020] (F1-Average=69.3%). El contraste entre los modelos zero-shot (S0, S3) y los <i>fine-tuneados</i> (S1, S2) evidencia que la detección de postura requiere adaptación supervisada por tópico: S0 colapsa a valores próximos a cero en cuatro de los cinco tópicos, mientras que S1 alcanza F1-Average>60% en todos salvo <i>climate</i> , tópico estructuralmente desequilibrado.	56

Índice de tablas

2.1	Ejemplos de análisis de sentimiento en tweets.	5
2.2	Ejemplos de postura (<i>stance</i>) en tweets sobre el Brexit.	6
2.3	Comparativa de rendimiento F1-Macro entre modelos basados en Transformers para Sentiment Analysis. Los 4 primeros modelos han sido evaluados en el benchmark de TweetEval, mientras que los dos últimos en los benchmarks mencionados (UMSAB-EN, que forma parte de SemEval-2017 y TASS 2020 respectivamente). Además, el último modelo, al ser evaluado en un benchmark en español, no es directamente comparable con los otros modelos en inglés.	8
2.4	Distribución de muestras y observaciones por target para <i>stance detection</i>	13
2.5	Resultados de F1-macro por tarea y modelo.	14
3.1	Corpus de pre-entrenamiento de los modelos de interés.	25
3.2	Estructura de la matriz de confusión para clasificación binaria.	26
4.1	Ventanas temporales usadas en el <i>scraper</i>	32
4.2	Tamaño de los splits de TweetEval stance tras el procesamiento.	39
4.3	Ejemplos representativos extraídos de los parquets procesados por campaña. La columna <i>BERTweet</i> muestra la normalización aplicada: @usuario→@USER, URL→HTTPURL. Los textos se muestran truncados a 150 caracteres. Normalización basada en [Nguyen et al., 2020]	42
4.4	Banco de modelos evaluados en la tarea de sentimiento. ZS = <i>zero-shot</i> ; FT = <i>fine tuning</i>	44
4.5	Estrategias de modelado para <i>Stance Detection</i> . ZS = <i>zero-shot</i> ; FT = <i>fine tuning</i>	45
4.6	Pesos de clase por subtópico (función de pérdida ponderada, train+val).	46
4.7	Pares campaña-tópico seleccionados para el análisis de postura	48
4.8	Eventos que hemos considerado clave anotados por campaña electoral	49
5.1	Resultados de los seis modelos evaluados en el test set de TweetEval Sentiment (n=12,284). Los modelos E1, E2 y E5 reportan el F1-Macro del mejor run (LR óptima, semilla óptima); la columna σ indica la desviación estándar sobre las tres semillas con esa LR. La referencia del paper es Barbieri et al. [2020] (F1-Macro=72,8%).	51
5.2	F1-Macro (%) en validación para cada combinación de tasa de aprendizaje y semilla aleatoria. En negrita la configuración seleccionada por modelo. $\bar{x}$ y σ calculados sobre las tres semillas.	51

5.3	F1 por clase (%) en el test set de TweetEval Sentiment para cada modelo (mejor configuración). La distribución de clases en test es: negativo 32,3 %, neutro 48,3 %, positivo 19,3 %.	52
5.4	Matriz de confusión de E0 (twitter-roberta-base-sentiment , sin fine-tuning adicional) sobre el test set de TweetEval Sentiment (n=12284). Filas = clase real; columnas = clase predicha.	53
5.5	F1-Average (%) por estrategia y tópico en el test set de TweetEval Stance. $F1_{\text{Average}} = (F1_{\text{against}} + F1_{\text{favor}}) / 2$ (métrica oficial SemEval-2016, excluye <i>none</i>). Referencia TweetEval [Barbieri et al., 2020]: 69,3 %. . . .	55
5.6	F1 por clase (%) de S1 y S2 (modelos fine-tuneados) por tópico. F1-Average es la métrica principal; F1-none se incluye solo como referencia. . .	55
5.7	Matrices de confusión de S1 (twitter-roberta-base fine-tuneado) por tópico sobre el test set. Filas = clase real; columnas = clase predicha. Clases: <i>none</i> (0), <i>against</i> (1), <i>favor</i> (2).	57

1 Introducción

El análisis de sentimiento a través de bases de datos obtenidas por medio de *scrapeo* de redes sociales es un campo de estudio relativamente reciente que ha empezado a generar interés público en los últimos años.

En particular, el análisis de sentimiento en Twitter (TSA) aborda el problema de analizar los mensajes publicados en esta plataforma en términos del sentimiento y las opiniones que expresan. A diferencia del análisis de sentimiento tradicional aplicado a reseñas de productos o artículos de noticias, el TSA presenta desafíos únicos derivados de las características particulares de Twitter que comentaremos en la siguiente sección [Giachanou and Crestani, 2016].

Este uso en concreto de las plataformas sociales por parte de los usuarios de expresar ideas, opiniones y sentimiento, ha generado una cantidad masiva de datos e información que representa una valiosa fuente de opinión pública que puede ser aprovechada en diversos contextos, especialmente en el ámbito político.

Durante las campañas electorales, millones de usuarios expresan sus posiciones, emociones y preferencias políticas en tiempo real, creando un ecosistema informativo sin precedentes. Sin embargo, este mismo ecosistema se ha convertido también en un terreno fértil para la manipulación política y la desinformación, donde actores malintencionados pueden influir en la opinión pública mediante el uso de bots, trolls y campañas coordinadas de información falsa.

Twitter (actualmente X) ha desempeñado un papel crucial en las campañas electorales modernas y será la fuente con la que más trabajaremos a lo largo de este trabajo. Desde las elecciones presidenciales de Estados Unidos en 2016, pasando por el referéndum del Brexit en Reino Unido, hasta las elecciones españolas de 2023, Twitter se ha consolidado como un espacio donde se libran batallas por la opinión pública. Estudios recientes han documentado casos de interferencia extranjera [Bovet and Makse, 2019], uso masivo de bots para amplificar mensajes políticos [Bessi and Ferrara, 2016], y campañas de desinformación sistemática [Shao et al., 2018] que han tenido impactos medibles en los resultados electorales.

1.1. El sentimiento en Machine Learning

El sentimiento se refiere a la actitud, emoción u opinión que expresa una persona hacia una entidad, tema o evento específico.

El análisis de sentimiento es la tarea del Procesamiento del Lenguaje Natural (PLN) que se encarga de la detección y clasificación de sentimientos en textos. La característica fundamental del sentimiento es su polaridad, que representa la orientación afectiva de

una opinión. Por lo general, los niveles considerados para clasificar la polaridad son:

- “positivo”: expresa aprobación, satisfacción o emociones favorables.
- “negativo”: expresa desaprobación, insatisfacción o emociones desfavorables.
- “neutro”: no expresa una orientación afectiva clara por lo que no es posible clasificar el sentimiento como positivo o negativo y/o presenta información objetiva sin aportar ningún tipo de juicio de valor.

Aunque en algunos casos se añaden categorías de granularidad fina (por ejemplo, “muy positivo” y “muy negativo”) o solo se tienen en cuenta las clases “positivo” y “negativo” [Balahur, 2013]. No obstante; para este trabajo, adoptaremos la clasificación ternaria (positivo, negativo, neutral) que es el estándar en la mayoría de estudios del campo.

Otra tarea relacionada con este tema es la detección de emociones, que se refiere a la clasificación de texto en varias clases de emociones, usualmente las básicas, según lo descrito por Paul Ekman [Ekman, 1992]. Aunque son diferentes en ciertos aspectos, parte de la investigación en este campo ha considerado estas tareas de forma conjunta bajo el concepto general (o “paraguas”) del análisis de sentimiento. Una tarea complementaria al análisis de sentimiento es la detección de postura (stance detection) [Mohammad et al., 2016], que tiene como objetivo identificar la posición del autor respecto a un target específico (una persona, tema o propuesta). A diferencia del análisis de sentimiento, que detecta la polaridad emocional general del texto (positivo, negativo, neutral), la detección de postura clasifica si el autor está a favor, en contra o no expresa una postura clara hacia una entidad concreta. Esta distinción resulta especialmente relevante en el análisis de campañas electorales, donde interesa conocer no solo el tono del debate (sentimiento) sino también el posicionamiento específico hacia candidatos o propuestas (postura). Aunque este trabajo se centra principalmente en el análisis de sentimiento, se explorará opcionalmente la detección de postura como análisis complementario en casos específicos como el Brexit, donde el debate se articuló claramente en torno a dos posiciones (Leave vs. Remain).

Algunos desafíos propios del análisis de sentimiento de textos son tratar con el sarcasmo, la negación y la ironía; saber identificar correctamente el sentimiento en función del contexto que puede estar presente de forma explícita o no en el texto, y hacer frente a las múltiples interpretaciones dadas por la ambigüedad.

Así, cuando en un texto aparece escrito por ejemplo “Trump es único”, ese “único”, ¿es un elogio o una crítica?

1.2. Twitter como fuente de datos

La elección de Twitter como fuente de datos para este trabajo responde a una serie de características únicas de la plataforma que la convierten en un recurso especialmente valioso para el análisis del discurso político y la opinión pública.

En primer lugar, desde su lanzamiento en 2006, Twitter ha atraído a cientos de millones de usuarios que publican más de 500 millones de mensajes diarios. Esta escala

masiva proporciona un volumen de datos suficiente para realizar análisis estadísticamente significativos sobre cualquier evento político.

En segundo lugar, a diferencia de plataformas como Facebook, donde la mayoría del contenido es privado, los tweets son accesibles públicamente en su mayor parte, lo que facilita la recopilación de datos para la investigación.

En tercer lugar y desde el punto de vista metodológico, los tweets presentan también ventajas significativas. Su estructura estandarizada incluye metadatos ricos como marcas temporales, geolocalización, hashtags, menciones y estadísticas de interacción (retweets, likes), que permiten enriquecer el análisis más allá del contenido textual. Los hashtags, en particular, han demostrado ser indicadores fiables de la relevancia temática de un tweet y permiten rastrear conversaciones específicas sobre candidatos, partidos o eventos electorales [Giachanou and Crestani, 2016].

Un factor determinante en la elección de Twitter como fuente de datos es la existencia de benchmarks y datasets académicamente validados construidos específicamente sobre esta plataforma. En este trabajo utilizamos TweetEval [Barbieri et al., 2020] como dataset de entrenamiento del modelo de clasificación de sentimiento. TweetEval es un benchmark de referencia en el campo del PLN para Twitter, desarrollado por el grupo CardiffNLP y disponible públicamente a través de la plataforma Hugging Face. Cabe destacar que TweetEval y los modelos preentrenados asociados están disponibles en múltiples idiomas, incluyendo inglés, español, árabe y francés, entre otros, lo que amplía considerablemente su aplicabilidad a contextos multilingües como los analizados en este trabajo, pues sabemos que existe una diferencia importante entre implementar un sistema que funcione para idiomas como el inglés (para el cual existen numerosos recursos lingüísticos y herramientas de análisis) y un sistema implementado para idiomas con pocas herramientas de este tipo, o uno que esté orientado a procesar datos de un gran conjunto de idiomas [Balahur, 2013]. De hecho, Twitter está disponible en más de 30 idiomas. Sin embargo, los usuarios “tweatean” en más de 80 idiomas diferentes.

Por último, es importante señalar, no obstante, que Twitter no es una muestra representativa de la población general. Su base de usuarios presenta sesgos demográficos hacia perfiles más jóvenes, urbanos y con mayor nivel educativo [Mislove et al., 2011].

Esta limitación, que será discutida en mayor detalle en las conclusiones, debe tenerse en cuenta al interpretar los resultados y extraer conclusiones sobre el comportamiento del electorado en su conjunto.

1.3. Motivación

El uso de Twitter como fuente de datos y de los tweets como elementos con los que trabajar para entrenar y validar nuestros modelos nos lleva obligatoriamente a adoptar una serie de medidas para hacer frente a peculiaridades propias de textos tan informales como son los tweets, ya que estos presentan características propias del lenguaje informal, como son: la brevedad (≤ 140 caracteres), el uso de @menciones, #hashtags, emoticonos y *slang*, junto con frecuentes errores ortográficos [Balahur, 2013]. Además, el contexto temático suele estar explícito en los hashtags, y en eventos de alta relevancia, la velocidad

de publicación supera los miles por minuto, lo que los hace útiles para análisis en tiempo real.

Siguiendo con este último punto, transmitir y consumir información de eventos o crisis a través de las redes sociales tiene sus consecuencias. Twitter, en particular, se ha convertido en un escenario central donde candidatos, partidos y ciudadanos libran batallas discursivas que pueden tener un impacto directo en los resultados electorales. Sin embargo, este mismo espacio ha demostrado ser vulnerable a fenómenos de manipulación informativa, como la difusión sistemática de noticias falsas.

Cuatro casos de estudio han sido determinantes en la comprensión de estos fenómenos. La campaña presidencial de Donald Trump en 2016, que reveló por primera vez el potencial de Twitter como instrumento de interferencia política [Bovet and Makse, 2019]; el referéndum del Brexit, celebrado ese mismo año en Reino Unido, que mostró cómo la polarización extrema en redes sociales y la formación de cámaras de eco podían fragmentar a una sociedad en torno a narrativas irreconciliables [Gorodnichenko et al., 2021], las elecciones españolas de 2023, que evidenciaron cómo la polarización ideológica en Twitter se intensifica en campañas multipartidistas, con narrativas antagónicas entre bloques que amplifican la fragmentación del discurso público [Marín Albaladejo and Navarro-Beltrá, 2025]; y, finalmente, las elecciones estadounidenses de 2024 y el ascenso de Donald Trump evidenciaron la evolución y sofisticación de estas técnicas.

Estos cuatro casos, distribuidos en un período de siete años y en contextos geopolíticos diferentes, ofrecen una oportunidad única para estudiar de forma comparativa cómo el sentimiento político se manifiesta en Twitter, cómo evoluciona a lo largo de una campaña y en qué medida refleja o anticipa los resultados electorales reales. A pesar de su relevancia, no existe hasta la fecha un análisis comparativo sistemático de estos cuatro casos que aplique una metodología PLN estandarizada y reproducible, lo que constituye el principal vacío que este trabajo pretende cubrir.

La motivación de este TFM es, por tanto, doble. Por un lado, contribuir al conocimiento científico sobre el papel de las redes sociales en la política contemporánea mediante el desarrollo y aplicación de técnicas de análisis de sentimiento y detección de postura. Por otro, ofrecer una metodología replicable que pueda extenderse a otros contextos electorales, aprovechando la disponibilidad de datasets públicos y herramientas de PLN de acceso abierto como TweetEval.

2 Estado del arte

Como ya hemos mencionado anteriormente, el análisis de la opinión abarca múltiples tareas de clasificación que, aunque relacionadas, persiguen objetivos distintos. Las dos tareas más relevantes en el contexto del análisis político son el análisis de sentimiento (*sentiment analysis*) y la detección de postura (*stance detection*). En esta sección se presentan ambas tareas y se discuten sus diferencias.

2.1. Tareas de clasificación de opinión en Twitter

2.1.1. *Sentiment Analysis* (Análisis de Sentimiento)

Como ya hemos mencionado en la introducción, el análisis de sentimiento, que es nuestra principal tarea, tiene como objetivo identificar la polaridad afectiva expresada en un texto. En su formulación más habitual, clasifica el texto en tres categorías [Liu et al., 2019]:

- Positivo: Expresa emociones, opiniones o valoraciones favorables
- Negativo: Expresa emociones, opiniones o valoraciones desfavorables
- Neutral: No expresa carga emocional clara o presenta información objetiva.

Ejemplos de esta clasificación ternaria:

Tweet	Sentiment	Justificación
“¡Qué gran día para la democracia!”	Positivo	Emoción favorable
“Esto es un desastre absoluto”	Negativo	Emoción desfavorable
“Las elecciones se celebran el domingo”	Neutral	Información objetiva

Tabla 2.1: Ejemplos de análisis de sentimiento en tweets.

2.1.2. *Stance Detection* (Detección de Postura)

La detección de postura tiene como objetivo identificar la posición del autor respecto a un target específico (una persona, organización, tema o propuesta). Requiere definir explícitamente el target y clasifica el texto en tres categorías [Mohammad et al., 2016]:

- Favor: El texto expresa apoyo, acuerdo o valoración positiva hacia el target

- **Against:** El texto expresa rechazo, desacuerdo o valoración negativa hacia el target
- **Neither:** El texto no expresa una postura clara, es neutral respecto al target, o no menciona el target de forma relevante

Ejemplos de esta clasificación ternaria para el target = “Brexit”:

Tweet	Stance	Justificación
“Votar Leave es lo mejor para UK”	<i>Favor</i>	Apoya el Brexit
“El Brexit destruirá la economía británica”	<i>Against</i>	Se opone al Brexit
“Hoy se vota el referéndum del Brexit”	<i>Neither</i>	No toma postura

Tabla 2.2: Ejemplos de postura (*stance*) en tweets sobre el Brexit.

A diferencia del *sentiment analysis*, *stance detection* requiere contexto: el mismo tweet puede clasificarse de forma diferente según cuál sea el target especificado.

Este TFM se centra principalmente en el análisis de sentimiento.

Sin embargo, opcionalmente, se explorará la tarea de *stance detection* como análisis complementario, permitiendo identificar no solo el tono emocional de los tweets, sino también el posicionamiento específico de los usuarios respecto a candidatos o propuestas concretas. Esto es especialmente relevante en el caso del Brexit, donde el debate se articuló en torno a dos posiciones claramente definidas (Leave vs. Remain).

La combinación de ambas tareas ofrece una visión más completa del discurso político en Twitter:

- *Sentiment analysis* → ¿Cuál es el tono del debate? (positivo/negativo/neutral)
- *Stance detection* → ¿A favor o en contra de quién/qué? (*favor/against/neither*)

2.2. Enfoques en TSA

El TSA ha experimentado un crecimiento notable desde los primeros trabajos realizados a finales de la primera década del siglo XXI. Se identifican cuatro grandes enfoques en la literatura [Giachanou and Crestani, 2016]:

1. Enfoques basados en Aprendizaje Automático:

Los métodos de aprendizaje automático construyen clasificadores a partir de conjuntos de tweets etiquetados. En el proceso se hace una recopilación y etiquetado de tweets, seguid de una extracción de características importantes, y, por último, el entrenamiento del clasificador [Giachanou and Crestani, 2016].

Entre los algoritmos de aprendizaje automático más utilizados en TSA se encuentran el Naive Bayes, los Support Vector Machines (SVM) y la regresión logística. Los SVM han demostrado consistentemente un rendimiento superior en tareas de clasificación de texto de alta dimensionalidad [Giachanou and Crestani, 2016].

2. Enfoques basados en Léxico:

Los métodos léxicos asignan una puntuación de sentimiento a un texto a partir de diccionarios de palabras etiquetadas con polaridad. Recursos como SentiWordNet [Esuli and Sebastiani, 2006] y el MPQA Opinion Corpus [Wilson et al., 2005] han sido ampliamente utilizados en TSA. La principal ventaja de estos enfoques es que no requieren datos de entrenamiento etiquetados; sin embargo, su principal limitación es la dificultad para adaptarse al lenguaje informal y en constante evolución de Twitter [Giachanou and Crestani, 2016].

3. Enfoques Híbridos:

Los enfoques híbridos combinan métodos de aprendizaje automático con léxicos de sentimiento para aprovechar las ventajas de ambos. Esta combinación mejora el rendimiento respecto a cada enfoque de forma individual [Giachanou and Crestani, 2016].

4. Deep Learning y Modelos de Lenguaje Preentrenados:

El desarrollo del Deep Learning ha supuesto una revolución en el TSA. Los modelos basados en redes neuronales recurrentes (LSTM) y, más recientemente, en arquitecturas Transformer han desplazado a los métodos clásicos en prácticamente todas las tareas de PLN.

El modelo BERT (Bidirectional Encoder Representations from Transformers) [Liu et al., 2019] marcó un punto de inflexión al introducir el preentrenamiento bidireccional sobre grandes corpus de texto. Su variante RoBERTa [Liu et al., 2019], con un preentrenamiento más robusto y sin la tarea de predicción de siguiente oración, mejoró aún más los resultados en tareas de clasificación de texto.

Para el dominio específico de Twitter, en [Nguyen et al., 2020] se desarrolló BERTweet, un modelo RoBERTa preentrenado sobre 850 millones de tweets en inglés. BERTweet ha demostrado rendimiento superior al de BERT y RoBERTa en múltiples tareas de TSA, incluyendo análisis de sentimiento, reconocimiento de entidades nombradas (NER) y etiquetado POS. Este modelo, junto con otros modelos específicos para Twitter, está disponible en la plataforma Hugging Face y es utilizado como base en el presente trabajo.

En particular, en el caso del idioma español, que nos interesa especialmente para la campaña de España 2023, [Pérez et al., 2022] presentaron un modelo conocido como RoBERTuito, preentrenado sobre más de 500 millones de tweets en español. Al ser evaluado sobre el benchmark TASS 2020, llegó a alcanzar un F1-Macro de 70.5%, superando así a modelos multilingüe genéricos. Es por esto y por su

capacidad de transferencia cruzada hacia tareas en inglés que nos puede interesar usarlo y ver cómo se comporta con los datos de TweetEval.

Los modelos con mejores resultados en la tarea de clasificación de sentimiento (según literatura) y que hemos mencionado brevemente en el estado del arte son los siguientes:

Modelo	F1-Macro	Fuente	Disponibilidad
twitter-roberta-base-sentiment	~72.8 % en	[Barbieri et al., 2020]	Hugging Face
BERTweet-base	~71.5 %	[Nguyen et al., 2020]	Hugging Face
RoBERTa-base (genérico)	~70.2 %	[Liu et al., 2019]	Hugging Face
BERT-base	~68.5 % (en benchmark UMSAB-EN)	[Devlin et al., 2019]	Hugging Face
twitter-xlm-roberta-base-sentiment	~70.6 % (en benchmark TASS 2020)	[Barbieri et al., 2022]	Hugging Face
robertuito-sentiment	~70.5 %	[Pérez et al., 2022]	Hugging Face

TABLA 2.3: Comparativa de rendimiento F1-Macro entre modelos basados en Transformers para Sentiment Analysis. Los 4 primeros modelos han sido evaluados en el benchmark de TweetEval, mientras que los dos últimos en los benchmarks mencionados (UMSAB-EN, que forma parte de SemEval-2017 y TASS 2020 respectivamente). Además, el último modelo, al ser evaluado en un benchmark en español, no es directamente comparable con los otros modelos en inglés.

2.3. Preprocesamiento en Twitter

Una de las particularidades del TSA es la necesidad de un preprocesamiento específico para el formato de los tweets. En [Giachanou and Crestani, 2016] se identifican cuatro categorías principales de características relevantes para el TSA:

- Características semánticas: palabras de opinión extraídas de léxicos de sentimiento, negaciones.
- Características sintácticas: n-gramas, unigramas, bigramas, etiquetas POS. Los unigramas y los bigramas le dicen al modelo qué palabras deben ir solas o en pareja. Las etiquetas POS son las que le dicen al modelo cómo funcionan esas palabras. Por ejemplo, ayuda a distinguir entre “Voto (sustantivo)” y “Voto (verbo)”.
- Características estilísticas: emoticones, intensificadores, mayúsculas, puntuación.
- Características específicas de Twitter: hashtags, menciones, retweets, URLs.

El preprocesamiento estándar para modelos basados en Transformers, establecido en [Barbieri et al., 2020], incluye la normalización de menciones de usuario (@usuario → @USER) y de URLs (http://... → HTTPURL), manteniendo los *hashtags* y los emojis por su valor informativo.

2.4. Benchmarks y Evaluación: TweetEval

La evaluación estandarizada ha sido un reto persistente en el campo del TSA, ya que la diversidad de datasets y métricas dificultaba la comparación entre métodos. Para abordar esta limitación, Francesco Barbieri, Jose Camacho-Collados, Leonardo Neves y Luis Espinosa-Anke propusieron TweetEval [Barbieri et al., 2020], un benchmark unificado que agrupa siete tareas de clasificación de tweets bajo un marco de evaluación común.

Las siete tareas de TweetEval son:

- Reconocimiento de emociones
- Detección de odio (*hate speech*)
- Detección de ironía
- Clasificación de emojis
- Análisis de sentimiento
- Detección de ofensas
- Detección de postura (*stance detection*).

Para la tarea de análisis de sentimiento, TweetEval adopta los datos del SemEval-2017 Task 4 [Rosenthal et al., 2017], que clasifica tweets en tres categorías de polaridad: positivo, negativo y neutral. La métrica de evaluación principal es el F1-score macro, que trata equitativamente todas las clases independientemente de su frecuencia.

TweetEval está disponible públicamente a través de Hugging Face y ha sido ampliamente adoptado como referencia en la comunidad investigadora. Los modelos del estado del arte alcanzan un F1-macro de aproximadamente 72-75 % en la tarea de sentimiento [Barbieri et al., 2020], siendo RoBERTa-base el modelo con mejor rendimiento reportado en el benchmark original.

2.4.1. Estructura del benchmark y formato de los datos

Cada una de las correspondientes tareas tiene su propio directorio de datos de entrenamiento y test. La organización general es la siguiente:

```

tweeteval/
├── README.md ...Documentación del benchmark
├── TweetEval_Tutorial.ipynb ..Notebook con
│   ejemplos
├── datasets/ .....Datasets de las 7 tareas
│   ├── sentiment/
│   ├── stance/
│   ├── emoji/
│   ├── emotion/
│   ├── hate/
│   ├── irony/
│   └── offensive/
├── evaluation_script.py Script de evaluación
│   oficial
└── predictions/ .....Predicciones de ejemplo
    (baseline)

```

Figura 2.1: Estructura de directorios y archivos del repositorio TweetEval.

El directorio `datasets/` contiene los datos etiquetados para cada una de las siete tareas. Cada subdirectorio sigue una convención de nomenclatura uniforme, con la excepción de `stance detection` que presenta una estructura especial que se detallará más abajo. El directorio `predictions/` incluye las predicciones del modelo RoBERTa-base re-entrenado en Twitter, correspondiente al mejor sistema evaluado en el paper original [Barbieri et al., 2020] como ya hemos mencionado, que sirve como baseline de referencia para comparaciones.

En cuanto al dataset de `sentiment`, este contiene aproximadamente 59899 tweets distribuidos entre los conjuntos de entrenamiento, validación y test. Se trata del segundo dataset más grande del benchmark TweetEval (tras `emoji prediction`), lo que proporciona una base sólida para el entrenamiento de modelos supervisados.

Estructura de archivos y organización El directorio `datasets/sentiment/` sigue la estructura estándar de TweetEval:

```
datasets/sentiment/  
├── train_text.txt ...Tweets de entrenamiento  
├── train_labels.txt ..... Etiquetas de  
│   entrenamiento  
├── val_text.txt ..... Tweets de validación  
├── val_labels.txt .... Etiquetas de validación  
├── test_text.txt ..... Tweets de test  
├── test_labels.txt ..... Etiquetas de test  
└── mapping.txt ..... Mapeo label_id a texto
```

Figura 2.2: Estructura interna de archivos del dataset sentiment para la tarea de *Sentiment Analysis*.

Cada uno de los archivos `_text.txt` contiene un tweet por línea, preservando el formato original sin aplicar ningún tipo de preprocesamiento. Esto incluye *hashtags*, menciones de usuario (en formato `@usuario`), URLs, emojis y cualquier otra característica propia del lenguaje de Twitter. Esta decisión de diseño permite que cada investigador aplique las técnicas de preprocesamiento que considere más apropiadas para su metodología.

Los archivos `_labels.txt` contienen una etiqueta numérica por línea, con correspondencia directa línea a línea respecto a los archivos de texto. Es decir, la etiqueta en la línea `n` del archivo `train_labels.txt` corresponde al tweet en la línea `n` del archivo `train_text.txt`.

En cuanto al formato de los datos y la codificación de las etiquetas para sentiment, el archivo `mapping.txt` establece la correspondencia entre los identificadores numéricos de las etiquetas y sus categorías semánticas:

0. negative

1. neutral

2. positive

Por su lado, el dataset completo para stance contiene aproximadamente 4163 tweets, lo que lo convierte en uno de los conjuntos de datos más pequeños del benchmark TweetEval.

A diferencia del resto de tareas de TweetEval, la detección de postura presenta una organización jerárquica basada en temas o targets específicos. Esto refleja la naturaleza intrínseca de la tarea: la postura siempre se define respecto a algo o alguien. Los temas de los que dispone la tarea para *stance* son:

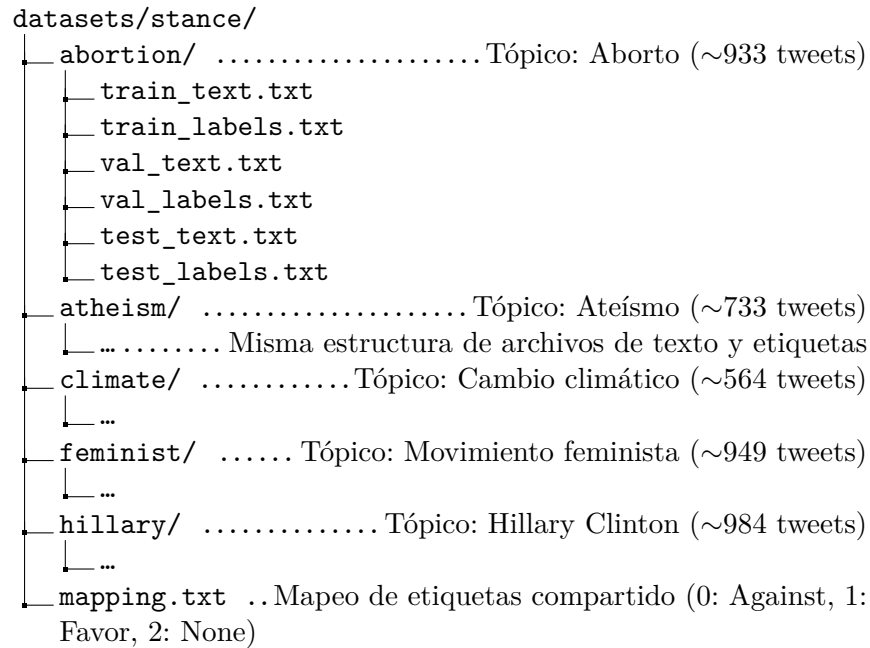


Figura 2.3: Organización jerárquica del dataset stance para la tarea de *Stance Detection* por tópicos.

Cabe mencionar que los cinco targets incluidos en TweetEval corresponden a temas políticamente polarizantes en el contexto estadounidense durante el período 2015-2016, aunque el debate alrededor de muchos de ellos actualmente sigue siendo relevante:

- Abortion (aborto): Debate sobre el derecho al aborto.
- Atheism (ateísmo): Postura respecto a la religión y el ateísmo.
- Climate change (cambio climático): Posicionamiento sobre la crisis climática.
- Feminist movement (feminismo): Opiniones sobre el movimiento feminista.
- Hillary Clinton: Postura específica respecto a la candidata presidencial.

Es importante destacar que el target “Hillary Clinton” es el único referido a una persona concreta, mientras que los demás corresponden a temas ideológicos. Esta heterogeneidad permite evaluar la capacidad de los modelos para generalizar la detección de postura tanto a entidades como a conceptos abstractos.

La distribución de tweets entre los diferentes targets es desigual, reflejando posiblemente la popularidad o controversia de cada tema en Twitter durante el período de recolección:

Target	Train + Val + Test	Observaciones
Hillary Clinton	~984	Mayor volumen, tema muy polarizante
Feminist movement	~949	Alto engagement en redes sociales
Abortion	~933	Debate perenne en política estadounidense
Atheism	~733	Menor volumen de conversación
Climate change	~564	El más pequeño del conjunto

Tabla 2.4: Distribución de muestras y observaciones por target para *stance detection*.

Esta variabilidad en el tamaño de los conjuntos de datos por target es una característica intrínseca del benchmark y debe tenerse en cuenta al entrenar modelos, especialmente si se pretende generalizar a través de múltiples targets.

En cuanto al formato, a diferencia de la estructura descentralizada por targets, el archivo mapping.txt es único y compartido por todos los subdirectorios:

- 0. none
- 1. against
- 2. favor

Debemos mencionar que el proceso de etiquetado de postura es inherentemente más complejo que el de sentimiento, ya que requiere: identificación del target, comprensión contextual para captar sarcasmo, ironía o referencias indirectas y evaluación de la intención del tweet. Es por eso que las métricas de rendimiento para las tareas de postura suelen ser inferiores a las de análisis de sentimiento.

Siguiendo con el tema de la evaluación, TweetEval proporciona un script de Python (evaluation_script.py) que implementa el protocolo de evaluación oficial del benchmark. Este script calcula las métricas apropiadas para cada tarea y reproduce exactamente los resultados reportados en el paper original. El script espera archivos de predicción con el mismo formato que los archivos _labels.txt.

De hecho, la carpeta predictions, que se puede ver en el esquema de la estructura, contiene las predicciones generadas por el modelo RoBERTa-base re-entrenado en 58 millones de tweets y del que hablaremos con más detalle en la sección de fundamentos teóricos. Estas predicciones sirven como baseline de referencia para comparaciones y corresponden al mejor modelo evaluado en [Barbieri et al., 2020].

Según los resultados reportados en el benchmark:

Tarea	F1-macro (%)	Modelo
Sentiment Analysis	71.3	RoBERTa-base
Sentiment Analysis	72.8	RoBERTa-Retrained
Stance Detection	68.0	RoBERTa-base
Stance Detection	69.3	RoBERTa-Retrained

Tabla 2.5: Resultados de F1-macro por tarea y modelo.

3 Fundamentos teóricos

3.1. Arquitecturas de modelos de lenguaje

Como ya hemos mencionado en el Estado del Arte, el desarrollo de modelos de lenguaje basados en arquitecturas de Deep Learning ha revolucionado el campo del PLN en los últimos años. En esta sección se presentan los conceptos fundamentales detrás de las arquitecturas utilizadas en este trabajo.

3.1.1. Transformers

Los transformadores son un tipo de arquitectura de red neuronal que transforma o cambia (de ahí su nombre) una secuencia de datos de entrada en una secuencia de datos de salida.

En el paper original que propuso inicialmente esta arquitectura, el transformer permitía realizar la traducción de un idioma a otro con la gran ventaja de poder entrenar al modelo en paralelo; lo que aumentaba drásticamente la velocidad y reducción del coste; y utilizando como potenciador el mecanismo de atención. Asimismo, permite que la red tenga memoria total: cada palabra del texto tiene acceso a las demás. Esto elimina el problema que presentaban las redes usuales de la memoria limitada, que hace que, en frases o textos muy largos, la red se vaya olvidando de las palabras del principio.

Con el tiempo, esta arquitectura resultó ser flexible y se pudo utilizar para tareas más allá del PLN, además de para la generación de texto, clasificación y resumen de contenidos, también pudo traspasar esa frontera y ser aplicado en Visión Artificial, Generación de Audio, Predicción de Series Temporales y Aprendizaje por Refuerzo.

De hecho, en los últimos años los Transformers se han puesto muy por encima de las Redes Neuronales Recurrentes (RNN) y de las redes convencionales en PLN, y para entender por qué debemos diseccionar su arquitectura. A diferencia de una red neuronal usual (como un Perceptrón Multicapa o MLP), el Transformer no trata los datos como una secuencia rígida ni como puntos independientes, sino como un grafo de relaciones [Pérez, 2020].

La arquitectura general de un transformer es como sigue:

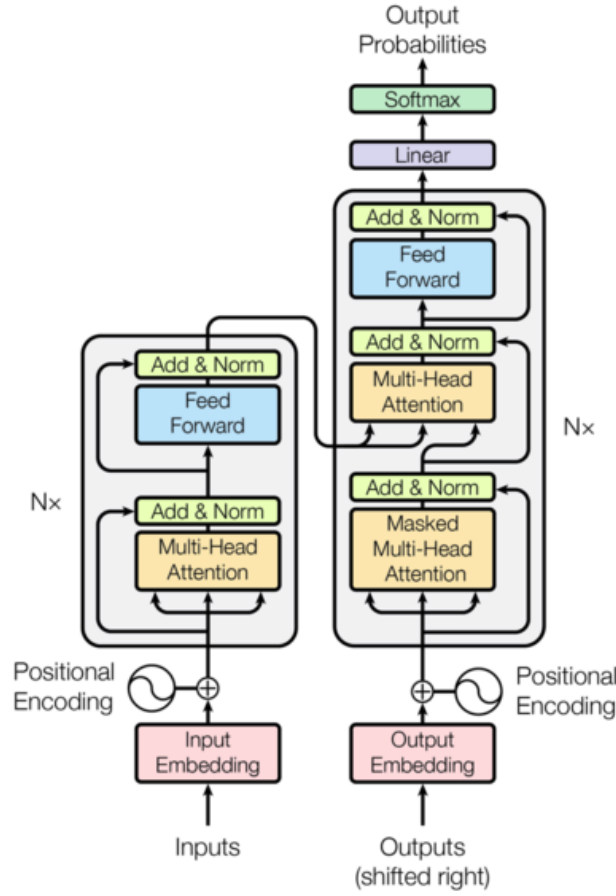


Figura 3.1: Arquitectura general del transformer. Adaptado de [Pérez, 2020]

Si nos fijamos en la Figura 3.1, podemos ver que hay 2 partes bien diferenciadas de la arquitectura. La caja de la izquierda, que se correspondería con el Encoder, y la caja de la derecha, que se correspondería con el Decoder. En su interior tanto Encoder como Decoder cuentan con un componente de Atención (o dos) y una red neuronal “normal” como salidas. Cabe destacar que para tareas de clasificación de sentimiento, como en nuestro caso, se suele utilizar únicamente el bloque Encoder.

Así, el corazón del Transformer es el Scaled Dot-Product Attention. Matemáticamente, se define mediante tres matrices: Queries (Q), Keys (K) y Values (V), derivadas de la entrada X mediante pesos aprendidos.

La salida de la atención se calcula como:

$$Attention(Q, K, V) = softmax(\frac{Q \cdot K^T}{\sqrt{d_k}})V$$

donde d_k es a dimensión de las claves o la dimensión de cada cabeza de atención. El factor

$\frac{1}{\sqrt{d_k}}$ previene que el producto escalar crezca excesivamente, lo que causaría gradientes pequeños en la función softmax.

El mecanismo de atención nos ayuda a crear y dar fuerza a las relaciones entre palabras en una oración.

Mirando de nuevo la Figura 3.1, podemos ver que en el momento de entrenar, estamos pasando el Input (que podría ser la frase a traducir o el twit que nos interesa clasificar en nuestro caso) y el Output (que podría ser la traducción del input o la etiqueta de sentimiento de nuestro twit) al mismo tiempo al modelo.

Tradicionalmente, usamos la salida únicamente para “validar” el modelo y ajustar los pesos de la red neuronal (durante el backpropagation). Sin embargo en el Transformer estamos usando la salida como parte del aprendizaje que realiza el modelo mediante una técnica llamada *Teacher Forcing*.

Entonces, es por eso que el output es también la entrada del Decoder hacia embeddings, posicionamiento y el ingreso a la Masked Self Attention que comentaremos a continuación.

Además, podemos diferenciar 3 tipos de atención distinta:

1. Self-Attention.
2. Cross-Attention.
3. Masked-Attention.

Self-Attention se refiere que crearemos los valores y las matrices de Q,K,V a partir de las propias entradas de los tokens de entrada.

En la Atención Cruzada, el Decoder recibe los valores (V) del Encoder para generar su representación de salida.

Y la llamada Masked-Attention se refiere a que enmascaramos la parte triangular superior de la matriz de atención al calcularla para no caer en *data leakage*, es decir, para evitar que el modelo acceda a tokens futuros durante el entrenamiento. En el Encoder, cuando tenemos los valores de entrada y hacemos self-attention, calculamos la atención de “todos contra todos” porque todos los tokens de entrada son conocidos desde el principio, pero cuando vamos produciendo el output y vamos escribiendo la cadena palabra por palabra, se supone que no podemos conocer el contenido de las palabras que aún no han sido generadas.

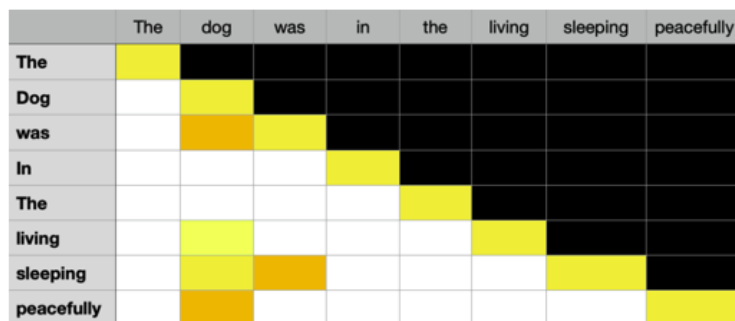


Figura 3.2: Resultado de la enmascaración de la matriz $Q \cdot K$ para una frase de ejemplo. Adaptado de [Pérez, 2020]

Para entender más en profundidad la arquitectura de estas redes, viene bien entender de qué otras capas está formada y cómo se van sucediendo. Las capas del transformer son:

a) Capa de Self-Attention Multi-Cabeza:

Cada palabra genera 3 vectores: Query (Q), Key (K) y Value (V). Se mide la afinidad entre Q y K de todas las palabras para decidir cuánto valor (V) aportar a la representación final. El modelo realiza este proceso en paralelo varias veces para capturar distintas relaciones o dimensiones.

b) Capa Forward:

Tras la atención, cada posición pasa por una red neuronal densa idéntica. Lo revolucionario es que esta capa se aplica a cada palabra de forma independiente y en paralelo, lo que permite aprovechar el rendimiento de la GPU. Su función es procesar la información capturada por la atención y proyectarla a un espacio de mayor dimensionalidad.

c) Capas de normalización y conexiones residuales:

Entre cada subcapa hay una conexión residual ($x + \text{Sublayer}(x)$). Esto permite que el gradiente fluya sin desvanecerse durante el entrenamiento de modelos muy profundos, evitando que la información original se pierda en el ruido de las transformaciones. La normalización previene que el rango de valores entre capas cambie “demasiado bruscamente” logrando hacer que el modelo entrene más rápido y con mayor capacidad de generalización.

d) Capa lineal y Softmax final:

La salida final del modelo pasa por una última capa Lineal y el Softmax. El Softmax nos dará un valor de entre 0 y 1 para cada una de las posibles palabras (tokens) de salida. Entonces si nuestro lenguaje total es de 50.000 posibles tokens (incluyendo signos de puntuación, admiración, interrogación), encontraremos a uno de ellos con valor más alto, que será la predicción final.

La parte más novedosa que introducen los transformers al campo del PLN es el *positional encoding* a los embeddings en los que convierte las palabras del input.

El *positional encoding* sirve para especificarle al modelo en qué orden o posición van los tokens que le pasamos. Porque está claro que, en el procesamiento de textos en particular, el orden de las palabras es crucial pues puede cambiar por completo el significado de la frase. No es lo mismo decir “el perro mordió al niño” que “el niño mordió al perro”.

Para resolver esto, se agrega una valor secuencial al embedding que se asume que la red podrá interpretar. Si aparece el token “perro” como segunda palabra o como quinta, mantendrá en esencia el mismo vector semántico pero con un ligero valor adicional que lo distinguirá y que será el que le dé la pista a la red neuronal de interpretar su posición en la oración.

Los creadores del Transformer propusieron utilizar funciones trigonométricas de diferentes frecuencias. La fórmula para una posición pos y una dimensión i del vector es:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

donde d_{model} representa la dimensión del modelo o el tamaño de la representación oculta.

Así, aunque el Transformer original nació para traducción (Encoder-Decoder) tal y como hemos mencionado antes, la tarea de análisis de sentimiento se beneficia principalmente del Encoder para generar representaciones ricas del contexto. Esto dio lugar a la familia de modelos pre-entrenados que son de los que vamos a hablar a continuación.

3.1.2. BERT: *Bidirectional Encoder Representations from Transformers*

Fue desarrollado por Google en 2018. Hoy en día, hay muchas versiones de BERT pre-entrenadas, pero en el artículo original, Google entrenó dos versiones de BERT: BERT-base y BERT-large con diferentes arquitecturas neuronales.

En esencia, BERTbase se desarrolló con 12 capas de transformación, 12 capas de atención y 110 millones de parámetros, mientras que BERTlarge utilizó 24 capas de transformación, 16 capas de atención y 340 millones de parámetros. Como era de esperar, BERTlarge superó al anterior en las pruebas de precisión [Devlin et al., 2019].

BERT supuso un cambio al eliminar la restricción de lectura de izquierda a derecha de los modelos anteriores.

Mientras que los modelos autorregresivos maximizan la probabilidad de una palabra dado su contexto izquierdo $P(w_i|w_1, w_2, \dots, w_{i-1})$, BERT utiliza el Masked Language Model (MLM). En este proceso, se enmascara aleatoriamente un porcentaje de los tokens de entrada (usualmente el 15 %) y el modelo debe predecirlos utilizando el contexto tanto a la izquierda como a la derecha simultáneamente.

Al enmascarar una palabra de una frase, esta técnica obliga al modelo a analizar las palabras restantes en ambas direcciones de la frase para aumentar las posibilidades

de predecir la palabra enmascarada. El MLM (*Masked Language Modeling*) se basa en técnicas ya probadas en el campo de la visión por ordenador, y es estupenda para tareas que requieren una buena comprensión contextual de toda una secuencia [Devlin et al., 2019].

Así, formalmente, si x es una secuencia y $\hat{x}$ es su versión enmascarada, BERT minimiza la pérdida de entropía cruzada:

$$L_{LML} = -\mathbb{E}_{x \sim D} \sum_{i \in m} \log(P(x_i | \hat{x}))$$

donde m es el conjunto de índices de los tokens enmascarados y la esperanza $\mathbb{E}_{x \sim D}$ (de que el tweet x sea aleatoriamente escogido del dataset D) indica que lo que nos importa es el promedio de error en todo el conjunto de datos. Es decir, que la pérdida total del modelo (L) es el promedio negativo de los aciertos del modelo sobre todos los tweets (x) que existen en D .

A diferencia de una métrica rígida como el Accuracy (donde o aciertas o fallas), la Entropía Cruzada es derivable. Esto permite que el modelo sepa qué tan lejos se ha quedado de la verdad. No es lo mismo fallar por poco (darle un 40 % a la respuesta correcta) que fallar por mucho (darle un 1 %). El gradiente de esta función guía a los pesos del Transformer a ajustarse de forma precisa.

Esta bidireccionalidad profunda es crítica en Twitter para entender sarcasmos o negaciones que dependen de palabras situadas al final de la oración.

Algunas de las variantes más conocidas del modelo BERT son:

- **RoBERTa:** Abreviatura de *Robustly Optimized BERT Approach*, RoBERTa es una variante de BERT creada por Meta en colaboración con la Universidad de Washington. Considerada una versión más potente que el BERT original, RoBERTa se entrenó con un conjunto de datos 10 veces mayor que el utilizado para entrenar el BERT. En cuanto a su arquitectura, la diferencia más significativa es el uso del aprendizaje de enmascaramiento dinámico en lugar del aprendizaje de enmascaramiento estático. Esta técnica, que consistía en duplicar los datos de entrenamiento y enmascararlos 10 veces, cada vez con una estrategia de enmascaramiento distinta, permitió a RoBERTa aprender representaciones más robustas y generalizables de las palabras [Liu et al., 2019].

Asimismo, se demostró que la tarea de *Next Sentence Prediction* (predecir si una frase sigue a otra) no aportaba valor significativo y se eliminó para centrar la capacidad del modelo en el lenguaje. Como última diferencia con el BERT original, mencionar que utiliza una versión de *Byte-Pair Encoding* (BPE), del que hablaremos más en profundidad y que trabaja con bytes en lugar de caracteres Unicode, lo que permite manejar un vocabulario más amplio y reducir los tokens desconocidos ([UNK]).

- **BERTweet: Especialización en el dominio de Twitter:** Como se observa en la Tabla 2.3, los modelos específicos para Twitter suelen superar a los genéricos.

BERTweet es el primer modelo de lenguaje a gran escala pre-entrenado específicamente para tweets en inglés (o multilingüe según la versión).

La diferencia fundamental reside en su corpus de entrenamiento (850M de tweets) y en su pre-procesamiento. Mientras que BERT se entrena con Wikipedia, BERTweet aprende la semántica de elementos propios de la red social:

- **Manejo de entidades:** Mejor comprensión de @usuarios, #hashtags y URLs.
 - **Emojis:** Los embeddings de BERTweet están diseñados para capturar la carga sentimental de los emojis, que en Twitter actúan como modificadores críticos del sentimiento.
- **DistilBERT:** Desde el lanzamiento de los primeros LLM a finales de la década de 2010, se ha consolidado la tendencia a construir LLM más grandes y pesados, tal y como se observa en la Figura 3.4. Esto tiene sentido, ya que parece haber una relación directa entre el tamaño del modelo y su precisión. Sin embargo, también es cierto que cuanto mayor es el modelo, más recursos necesita para funcionar y, por tanto, menos gente puede permitirse utilizarlo. DistilBERT pretende hacer más accesible el BERT ofreciendo una variante más pequeña, rápida, barata y ligera. Basándose en la arquitectura del BERT original, DistilBERT utiliza técnicas de destilación de conocimientos durante el pre-entrenamiento para reducir el tamaño en un 40 %, conservando el 97 % de sus capacidades de comprensión del lenguaje y siendo un 60 % más rápido.

Así, DistilBERT surge como una alternativa ligera mediante una técnica denominada *Knowledge Distillation* (Destilación de Conocimiento), basada en el popularmente conocido método de entrenamiento de profesor-alumno, donde un modelo pequeño (el Alumno) aprende a imitar el comportamiento de un modelo grande y pre-entrenado (el Profesor, en este caso BERT-base). De esta forma, en lugar de entrenar al alumno solo con las etiquetas reales (0 o 1), se le entrena para que su salida sea idéntica a la distribución de probabilidad del profesor. Esto es crucial porque el profesor “sabe” que una palabra puede ser un 90 % probable, pero otra palabra similar tiene un 8 % y una errónea un 2 %. Esa “incertidumbre rica” es lo que el alumno hereda.

Para lograr esto, se añade una componente de pérdida de destilación $L_{distill}$ a la función de entrenamiento, que se calcula usando la Divergencia de Kullback-Leibler (KL), para la que se cogen las predicciones del profesor con $T > 1$ y las predicciones del alumno con $T > 1$:

$$L_{distill} = \sum_i t_i \log(s_i)$$

donde t son las soft targets (las probabilidades del profesor) y s , las del alumno. Para suavizar estas probabilidades y que el alumno aprenda mejor las relaciones sutiles, se utiliza un hiperparámetro llamado Temperatura (T) en la función Soft-max, de forma que, si T es alta, la distribución es más “suave”, revelando qué

palabras considera el profesor que son “casi correctas”:

$$y_i(x|T) = q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

.

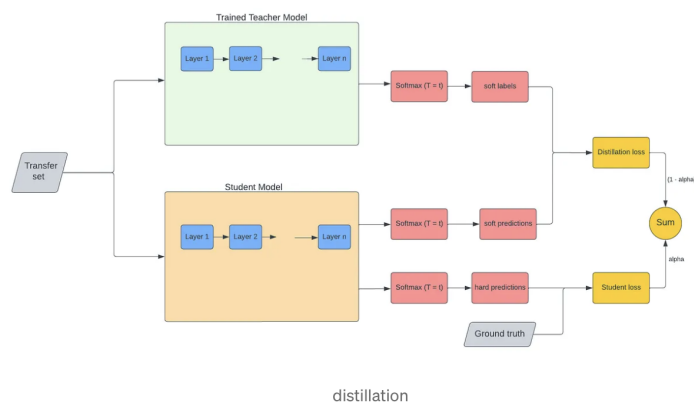


Figura 3.3: Entrenamiento de destilación, para el que no se usa el conjunto de entrenamiento entero sino un conjunto conocido como *transfer set*. Adaptado de [Mohan, 2022]

Así, la función de pérdida total se calcula como la suma:

$$L_{total} = (\alpha \cdot L_{student}) + ((1 - \alpha) \cdot L_{distill})$$

donde $\alpha \in [0, 1]$ es un factor de peso, y donde $L_{student}$ representa el error del modelo alumno al intentar predecir los datos reales. Se trata de la pérdida de MLM.

Volviendo al BPE o *Byte-Pair Encoding*, este nos ayuda a entender cómo estos modelos procesan el *slang* y los errores ortográficos, crucial para nuestra tarea en cuestión.

En particular, el gran desafío de Twitter es el problema de las Palabras Fuera de Vocabulario (OOV, *Out Of Vocabulary*). En un modelo tradicional, si el usuario escribe “bueníiiiiimo”, el modelo lo marca como un token desconocido ([UNK]), perdiendo toda la carga semántica. BPE soluciona esto mediante una segmentación por sub-unidades (subwords).

El algoritmo del BPE no se basa en reglas lingüísticas (como separar prefijos o sufijos), sino en estadística pura. El proceso de construcción del vocabulario sigue estos pasos:

1. **Inicialización:** El corpus se tokeniza a nivel de carácter. El vocabulario inicial parte de todos los símbolos únicos del corpus, ya sean letras, dígitos o emojis.
2. **Conteo de pares:** Se calcula la co-ocurrencia de cada par de tokens contiguos en el corpus completo.

3. **Fusión:** El par con mayor frecuencia (u, v) se combina en un único token uv , que se incorpora al vocabulario.
4. **Iteración:** Los pasos 2 y 3 se repiten hasta que el vocabulario alcanza el tamaño objetivo (por ejemplo, 50.000 entradas en el caso de RoBERTa).

La versión que utilizan RoBERTa y BERTweet del BPE es una evolución llamada Byte-level BPE, donde, en lugar de empezar con caracteres Unicode (que pueden ser miles, incluyendo todos los alfabetos y emojis), el algoritmo empieza con los 256 bytes posibles. Esto garantiza que el vocabulario base sea minúsculo (256 unidades) y que cualquier cadena de texto (incluyendo emojis complejos o caracteres especiales de Twitter) pueda ser representada sin generar nunca un token [UNK].

El BPE optimiza una función de coste implícita, pues si el vocabulario es muy pequeño (solo caracteres), las secuencias son muy largas y el modelo tiene dificultades para captar relaciones a larga distancia (la atención se dispersa), mientras que si el vocabulario es muy grande (palabras completas), la matriz de embeddings es inmanejable y hay mucha dispersión (sparsity). Así, el BPE encuentra el punto donde los tokens más frecuentes son palabras completas y los menos frecuentes son fragmentos con significado (morfemas), lo cual es ideal para la naturaleza ruidosa y creativa del lenguaje en Twitter.

Matemáticamente hablando, podemos verlo como que el BPE intenta minimizar la longitud de la secuencia de tokens manteniendo un vocabulario V manejable. Se puede ver como una búsqueda de la longitud de descripción mínima:

$$\min_V \left(\sum_{x \in D} |T(x, V)| \right)$$

donde D es el corpus de tweets; V , el vocabulario o tokens permitidos y $|T(x, V)|$ el número de tokens en los que se divide el tweet x usando el vocabulario V .

En la siguiente imagen podemos ver algunos de los modelos más famosos basados en transformers.

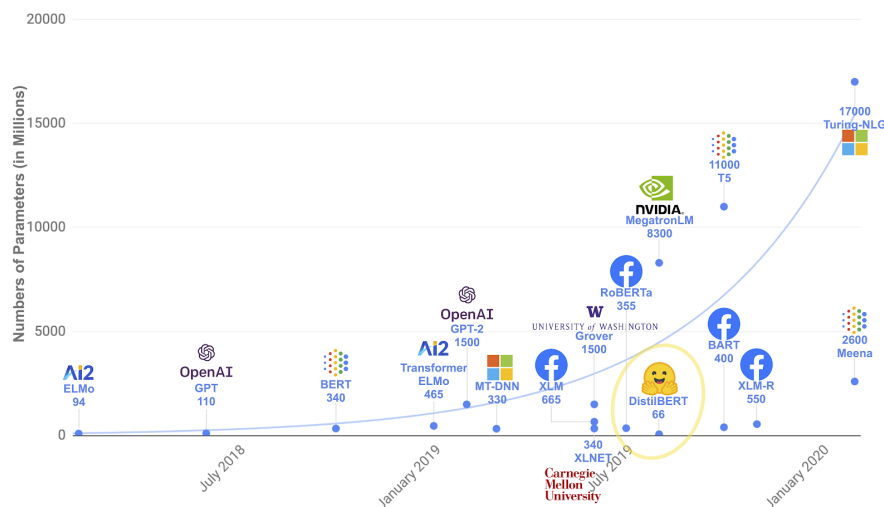


Figura 3.4: Evolución cronológica de los Modelos de Lenguaje basándose en su número de parámetros (en millones). Se observa una tendencia exponencial hacia modelos de gran escala, contrastando con versiones destiladas (DistilBERT) que buscan la eficiencia, lo cual en ocasiones puede ser clave para tareas de análisis en tiempo real, donde la inmediatez es lo primordial. Adaptado de [Pérez, 2020]

3.1.3. Fine-tuning de modelos preentrenados

Me gustaría comentar que, para este proyecto, no vamos a entrenar desde cero todos los modelos de la Tabla 2.3, sino que vamos a hacer un *fine tuning* o ajuste fino de los mismos.

La elección de este enfoque concreto se debe a distintos motivos: en primer lugar, este es el paradigma habitual en el estado del arte para tareas de clasificación de texto como las que nos traemos entre manos, ya que no disponemos de los recursos suficientes como para entrenar una arquitectura Transformer desde cero, como son, millones de datos disponibles y computación suficiente sin arruinarnos. Y, en segundo lugar, tal y como se ha visto en [Barbieri et al., 2020], se ha demostrado que el *transfer learning* o aprendizaje por transferencia resulta especialmente eficaz en este dominio, debido en parte a que los modelos internalizan semántica específica de Twitter o X, como son menciones, hashtags, emojis...

El *transfer learning* en PLN consiste en 2 fases. En una primera fase de pre-entrenamiento, el modelo aprende representaciones generales del lenguaje sobre corpus masivos sin supervisión. Algunos de estos corpus, que hemos visto en clase son por ejemplo la Wikipedia, el BookCorpus o CommonCrawl. Después, en la segunda fase, que es la de *fine tuning*, los pesos pre-entrenados se ajustan sobre un conjunto de datos etiquetado y específico para la tarea, en nuestro caso, TweetEval, añadiendo una cabeza de clasificación al modelo.

Algunos apuntes teóricos acerca del *fine tuning* son que la tasa de aprendizaje o *learning rate* durante esta fase debe ser varios órdenes de magnitud inferior que la usada durante el pre-entrenamiento, típicamente en el rango $[10^{-5}, 5 \times 10^{-5}]$, ya que una demasiado alta puede destruir las representaciones previamente aprendidas (esto se conoce como *catastrophic forgetting*); y que la función de pérdida estándar es la entropía cruzada:

$$\mathcal{L} = - \sum_{c=1}^C y_c \log(\hat{y}_c) \quad (3.1)$$

donde y_c es la etiqueta real de la clase c e $\hat{y}_c$ la probabilidad predicha. Cuando el corpus presenta desequilibrio de clases, esta pérdida puede adaptarse asignando un peso w_c inversamente proporcional a la frecuencia de cada clase.

A continuación se muestra una tabla con los corpus de pre-entrenamiento de la fase 1 para los distintos modelos mencionados en la Tabla 2.3 que más tarde usaremos sobre nuestros datos.

Modelo	Corpus de pre-entrenamiento	Referencia bibliográfica
twitter-roberta-base-sentiment	en torno a 58M tweets en inglés (ene 2018–dic 2021), proyecto TimeLMs.	[Loureiro et al., 2022]
twitter-roberta-base	mismo corpus	mismo paper
bertweet-base	en torno a 850M de tweets en inglés (2012-2018), 80GB de texto.	[Nguyen et al., 2020]
xlm-roberta-base	2,5TB CommonCrawl filtrado en 100 idiomas (texto general web, no Twitter).	[Conneau et al., 2020]
robertuito	en torno a 500M de tweets en castellano.	[Pérez et al., 2022]
bert-base-uncased	BookCorpus y Wikipedia inglesa.	[Devlin et al., 2019]

Tabla 3.1: Corpus de pre-entrenamiento de los modelos de interés.

3.2. Métricas de evaluación

La evaluación de los modelos de clasificación de sentimiento y detección de postura requiere el uso de métricas estandarizadas que permitan cuantificar su rendimiento y compararlo con el estado del arte. En esta sección se presentan las métricas utilizadas en este trabajo.

3.2.1. Métricas Básicas de Clasificación

Para entender las métricas de evaluación empleadas, es necesario introducir primero los conceptos básicos de matriz de confusión. Para una tarea de clasificación binaria (por ejemplo, positivo vs. negativo, o clase 0 vs. clase 1), la matriz de confusión contabiliza:

- TP (*True Positives*): Instancias positivas correctamente clasificadas como positivas.
- TN (*True Negatives*): Instancias negativas correctamente clasificadas como negativas
- FP (*False Positives*): Instancias negativas incorrectamente clasificadas como positivas
- FN (*False Negatives*): Instancias positivas incorrectamente clasificadas como negativas

Estas relaciones se organizan visualmente en una matriz de confusión, como se muestra en la Tabla 3.2:

		Valor Predicho	
		Positivo	Negativo
Valor Real	Positivo	TP	FN
	Negativo	FP	TN

Tabla 3.2: Estructura de la matriz de confusión para clasificación binaria.

A partir de estos valores se calculan las métricas fundamentales:

- **Precisión**

Mide la proporción de predicciones positivas que fueron correctas:

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall (Exhaustividad)**

Mide la proporción de instancias positivas reales que fueron detectadas:

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-Score**

Es la media armónica de precision y recall, proporcionando un balance entre ambas:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **Accuracy (Exactitud)**

Proporción de predicciones correctas sobre el total:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

3.2.2. F1-Score Macro: Métrica Principal

En tareas de clasificación multi-clase, como el análisis de sentimiento con tres categorías (positivo, negativo, neutral), existen diferentes formas de agregar las métricas de cada clase:

- F1-Micro: Calcula métricas globales agregando TP, FP y FN de todas las clases.
- F1-Macro: Calcula el F1 de cada clase por separado y luego promedia.
- F1-Weighted: Promedio ponderado por el número de instancias de cada clase. Para este trabajo se utiliza el F1-Score Macro como métrica principal, siguiendo el estándar establecido por TweetEval [Barbieri et al., 2020]. El F1-macro se calcula como:

$$F_1^{\text{macro}} = \frac{1}{C} \sum_{i=1}^C F_1^i$$

donde C es el número de clases y F_1^i es el F1-score de la clase i .

El F1-Macro trata todas las clases equitativamente, independientemente de su frecuencia en el dataset. Penaliza el rendimiento pobre en clases minoritarias, evitando que modelos sesgados hacia la clase mayoritaria obtengan puntuaciones artificialmente altas. Además, es más informativo que la accuracy en datasets desbalanceados, que es el caso habitual en análisis de sentimiento político donde la clase neutral suele ser minoritaria.

Por ejemplo, en un dataset con distribución 70% positivo, 20% negativo, 10% neutral, un clasificador que predice siempre “positivo” tendría 70% accuracy pero $F1\text{-macro} = 23\%$ F1-macro refleja mejor la capacidad real del modelo para discriminar entre clases.

3.2.3. F1-Average para *Stance Detection*

Para la tarea de detección de postura (*stance detection*), se adopta la métrica definida en SemEval-2016 Task 6 [Mohammad et al., 2016]. En esta tarea, los tweets se clasifican en tres categorías respecto a un target (candidato, tema):

- *Favor*: El tweet apoya al target.
- *Against*: El tweet se opone al target.
- *Neither*: El tweet es neutral o no expresa una postura clara.

Sin embargo, desde el punto de vista del análisis político, las clases de mayor interés son *Favor* y *Against*, ya que representan posicionamientos claros. Por este motivo, [Mohammad et al., 2016] definieron la métrica F1-average como:

$$F_1^{\text{avg}} = \frac{F_1^{\text{favor}} + F_1^{\text{against}}}{2}$$

Esta métrica excluye la clase *Neither* del cálculo, concentrando la evaluación en la capacidad del modelo para distinguir entre posturas a favor y en contra. Esto es especialmente relevante en campañas electorales, donde el objetivo es identificar el apoyo u oposición hacia candidatos específicos, más que detectar tweets neutrales [Mohammad et al., 2016].

El uso del F1-avg nos va a permitir enfatizar las clases de mayor interés analítico (*favor/against*), evitar que una buena clasificación de *neither* (clase mayoritaria) infle artificialmente las métricas y nos va a permitir realizar una comparación directa con trabajos previos en *stance detection* que usan SemEval-2016.

4 Metodología

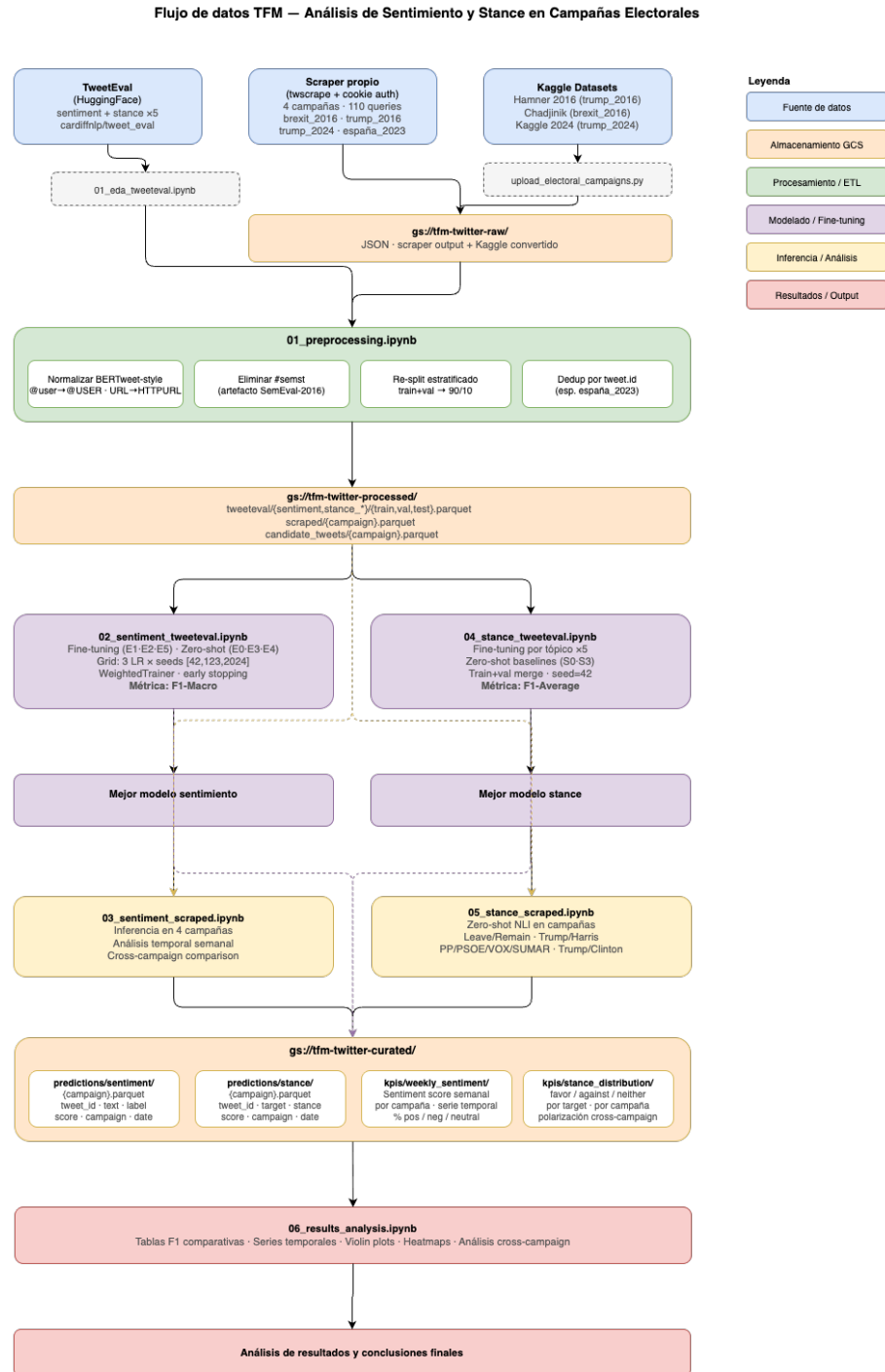


Figura 4.1: Flujo completo de los datos a lo largo de todo el proyecto. Imagen de generación propia.

4.1. Estrategia de recolección de datos propios

Aunque el benchmark TweetEval proporciona una base sólida para la evaluación estandarizada de modelos de clasificación de tweets, presenta una limitación fundamental para los objetivos de este trabajo, y es que sus datasets no contienen datos de las campañas electorales reales que nos interesan. Por este motivo, este trabajo adopta una estrategia en dos niveles: (1) evaluar y seleccionar modelos utilizando TweetEval como benchmark de referencia, garantizando comparabilidad con el estado del arte, y (2) aplicar los modelos seleccionados a datos propios recolectados específicamente para cada campaña electoral, que constituyen el verdadero objeto de análisis del trabajo.

Para la recolección de los datos, la idea original era hacer uso de la API oficial de la plataforma X, que permitía el acceso gratuito para fines educativos o de investigación. No obstante, a partir de 2023, se eliminó la vía gratuita y se discontinuó el Academic Research Access, y debido a la imposibilidad de pagar 5000 \$ para obtener el Full-Archive Search, hemos optado por construir una herramienta personal de *scraping* haciendo uso de la librería de código abierto twscrape (versión $\geq 0.14.0$), que permite recolectar tweets sin necesidad de acceso a la API oficial.

Desde el punto de vista ético, la recolección se restringe exclusivamente a tweets públicos, es decir, publicados por cuentas sin restricción de privacidad, en línea con las directrices de la Asociación de Internet de Investigación (AoIR) para la investigación ética en redes sociales [Association of Internet Researchers, 2012]. Los datos recopilados se utilizan únicamente con fines de investigación académica y no se comparten públicamente en su forma original para respetar los términos de servicio de la plataforma.

Estos datos recopilados se almacenan en Google Cloud Storage (GCS) en formato JSON, siguiendo una jerarquía de directorios que refleja la estructura del corpus: campaña/tarea/target/timestamp.json. Cada fichero contiene un array JSON con todos los tweets recuperados para una consulta determinada, donde cada tweet se representa como un objeto con los siguientes campos: identificador único, texto completo, fecha de publicación, identificador y nombre de usuario del autor, idioma detectado, contadores de interacción (*likes*, retweets, respuestas, citas), indicador de retweet, *hashtags* extraídos, y metadatos de la consulta que originó su recopilación (campaña, tarea, target, slug de consulta). Este formato es directamente compatible con la librería pandas de python para su posterior carga y uso.

En cuanto a la construcción de las consultas de búsqueda en el propio Twitter, estas constituyen una decisión metodológica crítica, ya que determinan directamente la relevancia temática y la representatividad del corpus recopilado. Se han diseñado un total de 110 consultas distribuidas entre las dos tareas (análisis de sentimiento y detección de postura) para cada una de las cuatro campañas electorales. Todas las consultas siguen la sintaxis avanzada de búsqueda de Twitter, combinando los siguientes operadores:

- Términos temáticos como *hashtags* canónicos, nombres de candidatos y partidos, términos de debate político específicos de cada campaña.
- Restricción temporal usando operadores *since* y *until* para acotar la búsqueda al

período de campaña activa de cada evento electoral.

- Un filtro de idioma para garantizar la coherencia lingüística del corpus (inglés para Brexit 2016, Trump 2016 y Trump 2024; español para España 2023).
- Exclusión de retweets, con el objetivo de centrarse en expresiones originales del usuario y evitar la inflación artificial del corpus por amplificación.
- Ventanas temporales. Los períodos de *scraping* se han definido para capturar la fase de mayor actividad política en cada campaña:

Campaña	Inicio	Fin
Brexit 2016	2016-04-15	2016-06-30
Trump 2016	2016-09-01	2016-11-15
España 2023	2023-06-20	2023-07-25
Trump 2024	2024-09-01	2024-11-10

Tabla 4.1: Ventanas temporales usadas en el *scraper*.

Las etiquetas para cada tweet serán inferidas a posteriori por un modelo de lenguaje (LLM). El procesamiento y etiquetado de los datos, los llevará a un segundo bucket de Google Cloud, de transformación, con la idea de seguir la línea bronce-plata-oro (crudo-procesado-curado) en cuanto a las capas de almacenaje de datos, que es la estándar en la ciencia de datos.

Todo el código relacionado con el *scraper* y el procesado y etiquetado de los datos se puede encontrar en mi github personal: <https://github.com/jimenami/TFM>.

Los buckets, por su parte, son de acceso privado por lo que hemos comentado antes de la privacidad.

Asimismo, decidí incorporar conjuntos de datos adicionales obtenidos de repositorios públicos en la plataforma Kaggle, con el fin de enriquecer el corpus de análisis con fuentes independientes al *scraping* propio.

Para la campaña estadounidense de 2016, se utilizó el dataset de [Hamner, 2016], que comprende 6.248 tweets publicados desde las cuentas oficiales @realDonaldTrump y @HillaryClinton entre enero y septiembre de 2016. El dataset incluye metadatos completos: identificador único del tweet, marca temporal, recuento de retweets y favoritos, e indicador de retweet. A diferencia de los datos propios scrapeados, este corpus recoge la ideología directa los propios candidatos, constituyendo una fuente complementaria que permite contrastar el discurso oficial con el sentimiento popular detectado. Tras filtrar por idioma inglés, el corpus resultante contiene 3.131 tweets de Trump y 3.117 de Clinton.

Para la campaña del Brexit de 2016, se incorporó el dataset de [Hadjinikolis, 2020], compuesto por 53.576 tweets publicados por parlamentarios y figuras políticas británicas hasta el 19 de junio de 2016, cuatro días antes del referéndum. Este dataset incluye

etiquetas de posicionamiento (*Stay/Leave*) asignadas a nivel de autor, lo que lo convierte en la única fuente del corpus con anotación de stance de referencia para datos propios de campaña.

Para la campaña presidencial estadounidense de 2024, se empleó un dataset de tweets públicos [Roy, 2025] recopilados entre junio y noviembre de 2024, que abarca el período completo de campaña. Con 26.425 tweets de usuarios anónimos, este corpus captura la conversación ciudadana en torno a la contienda Trump-Harris en un formato crudo compatible con el pipeline de normalización BERTweet.

En cuanto a las campaña española de 2023, no pude encontrar datos en kaggle aprovechables, ya que localicé un dataset pero su cobertura temporal no abarcaba la de la campaña. Para esta campaña, los datos scrapeados propios constituyen la única fuente disponible.

Los datos recopilados mediante *scraping* propio y mediante búsqueda en sitios públicos como Kaggle, cumplen en este trabajo una función fundamentalmente inferencial y analítica, no predictiva. A diferencia de los datasets de TweetEval, que proveen ejemplos etiquetados para entrenar y evaluar modelos en condiciones de benchmark controlado, el corpus scrapeado constituye el objeto de estudio real pues está formado por las conversaciones políticas que nos interesan y en “activo”, es decir, en el momento de llevarse a cabo dichas campañas.

Una vez validados los modelos sobre TweetEval, se aplican sobre este corpus para inferir la distribución de sentimiento y posicionamiento (*stance*) en cada campaña. El objetivo no es predecir un resultado futuro, sino describir y comparar echando la vista atrás los patrones de opinión pública expresados en Twitter, es decir, la evolución temporal del sentimiento hacia los candidatos, la polarización discursiva entre bloques, y las diferencias cross-campaign en intensidad y dirección del posicionamiento ciudadano.

Esta distinción es metodológicamente relevante porque el análisis se apoya en los dos tipos de datos con funciones distintas. Los datos de TweetEval sirven para evaluar y validar el modelo de análisis de sentimiento, pues son tweets ya etiquetados por humanos que permiten medir si el modelo clasifica bien; mientras que los datos scrapeados, son los que realmente importan para la investigación ya que son tweets reales sobre las campañas electorales que el modelo analiza para extraer conclusiones. Sin los datos scrapeados no hay análisis, y sin TweetEval no hay garantía de que ese análisis sea fiable.

4.2. Análisis exploratorio de los datos de TweetEval

Con la estructura del benchmark, que ya hemos estudiado en la sección 2.4.1, podemos explorar un poco más los datos de TweetEval para saber bien cómo se distribuyen en los dos conjuntos de sentimiento y postura. Esta exploración, correspondiente a la primera fase del flujo de la Figura 4.1, precede al procesamiento y sirve para fundamentar las decisiones metodológicas descritas en la sección siguiente. Se ha realizado a través del notebook de Jupyter `01_edu_tweeteval.ipynb`.

El conjunto de datos de sentimiento de TweetEval consta de 59.899 tweets distribuidos en tres particiones: entrenamiento (45.615), validación (2.000) y test (12.284). La

Figura 4.2 muestra la distribución de las tres clases (negativo, neutro y positivo) en cada partición.

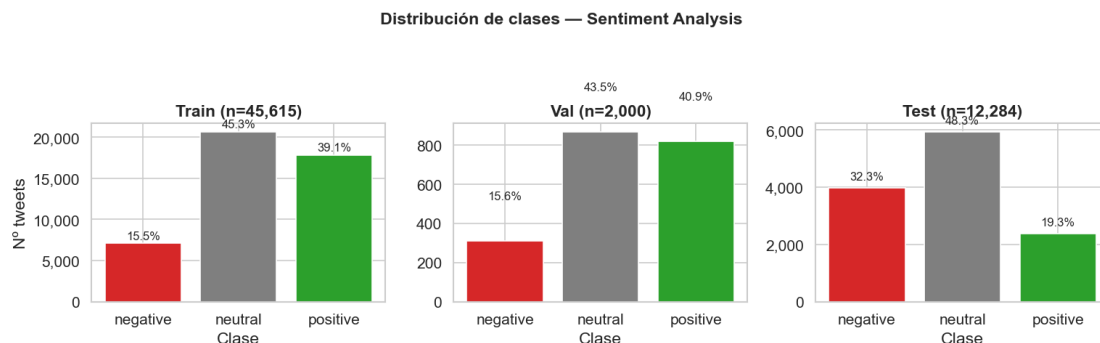


Figura 4.2: Distribución de clases por split en el dataset de sentiment.

El primer aspecto destacable y que ya hemos mencionado anteriormente es el desequilibrio de clases, que afecta de forma diferente a cada partición. En entrenamiento y validación, la clase dominante es la neutra (45,3 % y 43,5 % respectivamente), seguida de la positiva (39,1 % y 40,9 %) y, muy por detrás, la negativa (15,5 % y 15,6 %).

En cuanto a la longitud de los tweets, la Figura 4.3 muestra que la mayoría oscila entre 15 y 25 tokens, con una longitud media de 19,2 tokens en entrenamiento y un percentil 95 de 27 tokens. El conjunto de test presenta una longitud media ligeramente inferior (14,9 tokens), coherente con el hecho de que incluye tweets más variados, algunos de los cuales corresponden a periodos anteriores con el límite de 140 caracteres más restrictivo. El problema es que como las 3 clases siguen la misma distribución, aproximadamente normal, la longitud de los tweets no nos va a servir como feature discriminativo.

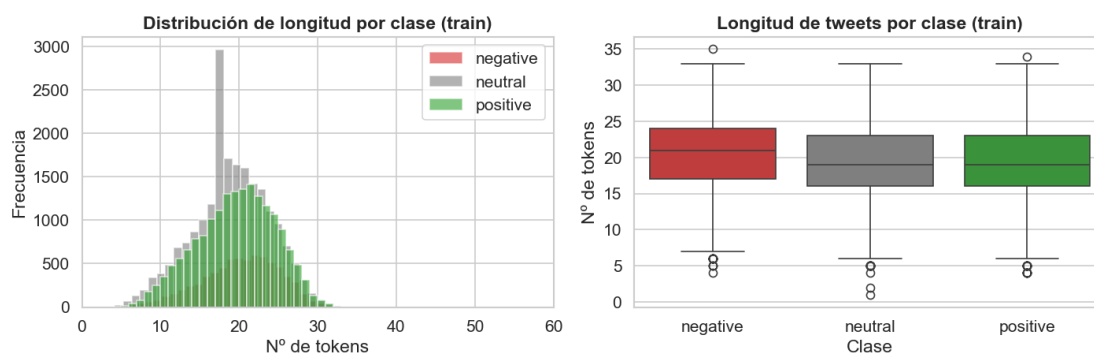


Figura 4.3: Distribución de longitud de los tweets en el conjunto de entrenamiento en el dataset de sentimiento.

En las figuras 4.4 y 4.5 mostramos respectivamente el top 15 tokens más frecuentes diferenciando por clase y eliminando stopwords, y el top 20 hashtags más frecuentes.

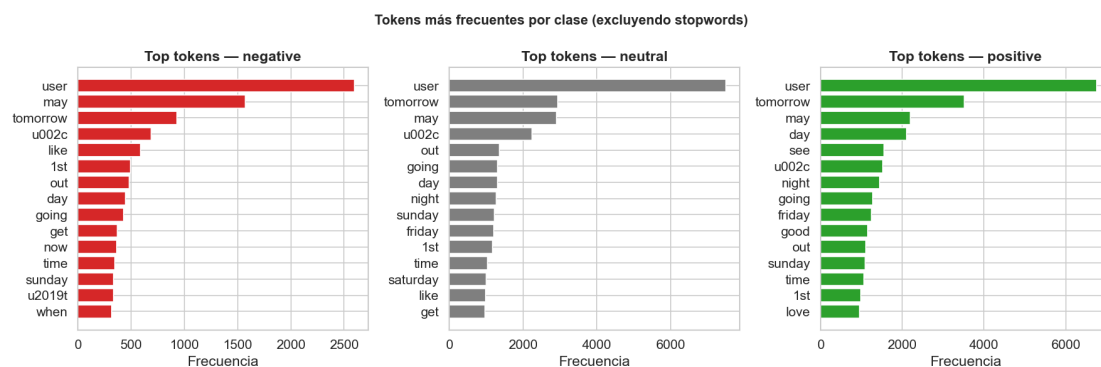


Figura 4.4: Tokens más frecuentes en el dataset separando por clase.

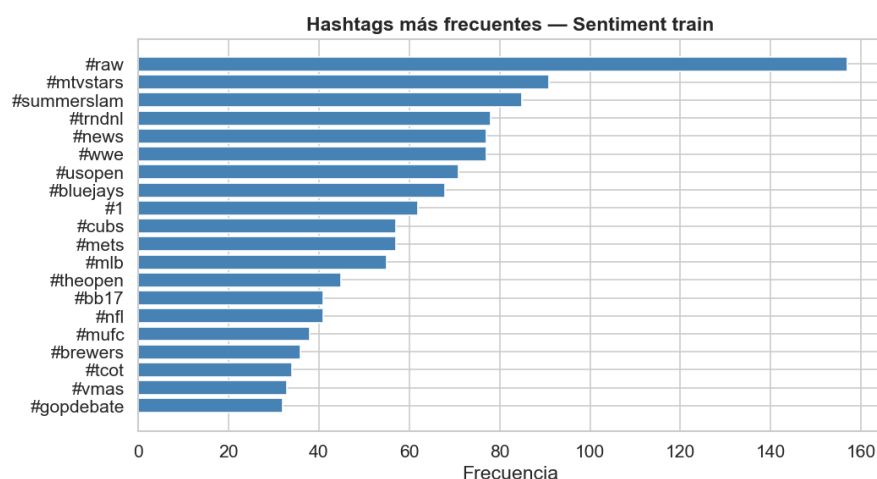


Figura 4.5: *Hashtags* más frecuentes en el dataset de sentimiento.

En el caso de los tokens, se aprecia que dominan las palabras propias del preprocesamiento, como *user* que sería el resultado de la anonimización del usuario correspondiente, o *u002c* que es la coma en formato Unicode. Esto pone de manifiesto que convendría limpiar el dataset.

Por otro lado, en las 3 clases hay presencia de palabras relacionadas con el tiempo, como los días de la semana, *sunday*, *friday*, *night*, *day*, *tomorrow*, *saturday*, *may*, *time* ... También hay palabras exclusivas de la clase positiva, como *good*, *love*, pero no exclusivas de la clase negativa.

En el caso de los *hashtags*, *#raw*, *#mtvstars*, *#summerslam*, *#wwe*, *#mlb* y *#cubs* nos hacen pensar que el corpus es mayoritariamente de deporte y entretenimiento, no político. Solo aparece *#gopdebate* (que hace referencia al debate sobre el *Grand Old Party* americano, es decir, el partido republicano) y *#tcot* (acrónimo para *Top Conservatives On Twitter*). Para aplicación electoral va a haber un hueco de dominio significativo que tendremos que solventar de alguna forma.

Paralelamente, hemos realizado otro EDA para el dataset de detección de postura y también hemos podido extraer varias conclusiones.

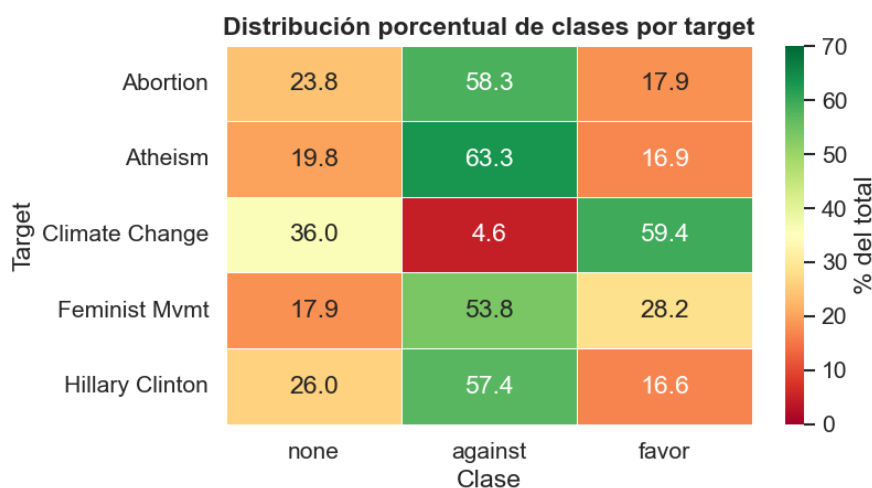


Figura 4.6: Distribuciones porcentuales a favor/neutral/en contra para los distintos tópicos del dataset

Como muestra la Figura 4.6, en 4 de los 5 tópicos la mayoría del dataset se muestra en contra, a excepción del cambio climático, donde la mayoría se muestra a favor (entendemos este a favor como “a favor de que es un problema real, existe y no se puede ni debe negar”).

Hemos estudiado la distribución por clase en el split train-val-test y nos hemos dado cuenta de que el conjunto de validación es muy pequeño para todos los tópicos, de entre 40 y 70 muestras. Esto nos indica que va a hacer falta aplicar algún método de combinación entre el conjunto de validación y el de test o usar la validación cruzada para así evitar la alta varianza en los resultados. Para todos los tópicos, no obstante, las distribuciones son muy similares.

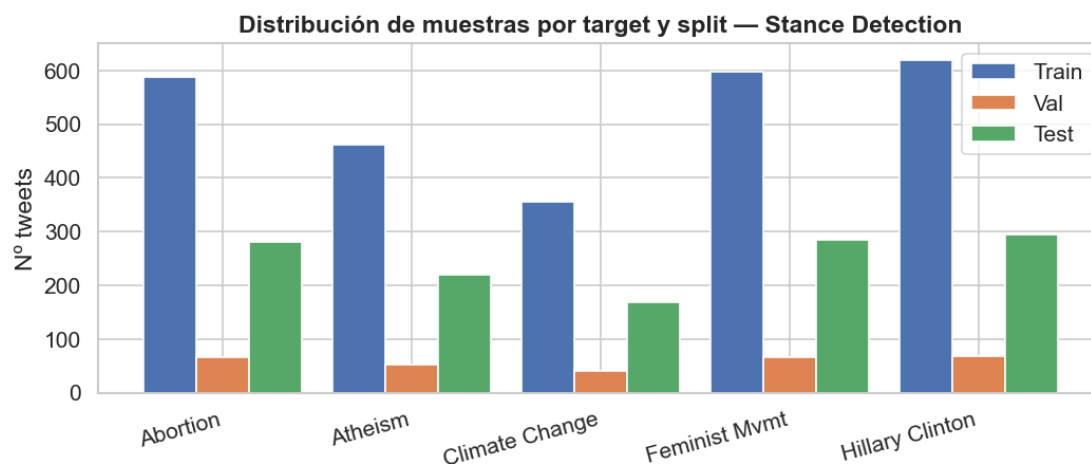


Figura 4.7: Distribuciones del split para los distintos tópicos del dataset de *Stance Detection*.

En cuanto a la longitud de los tweets, no varían especialmente, de media, en función del tópico y, en general, los tweets se siguen manteniendo en una media de extensión de entre 15 y 20 tokens, similar a lo que pasaba en el dataset de sentimiento.

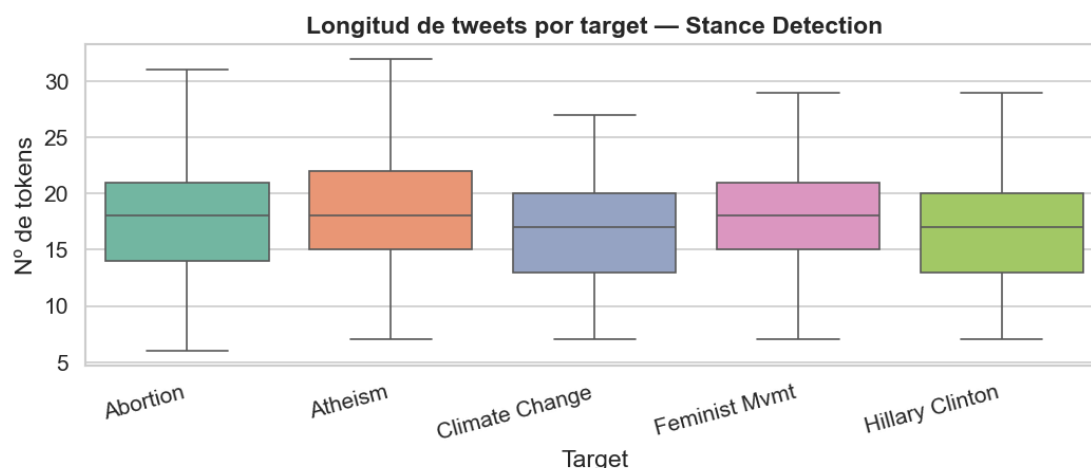


Figura 4.8: Longitud de los tweets para los distintos tópicos del dataset stance.

Por último, y en relación al vocabulario empleado en los tweets de este dataset, cabe destacar que sí que hay presencia de vocabulario específico para los distintos tópicos, por lo que esas palabras sí que podrían utilizarse como una feature específica y discriminadora a la hora de clasificar usando los modelos de lenguaje. Destacan:

- Abortion: *abortion, life, women, babies, unborn.*
- Atheism: *god, jesus, lord, believe, religion.*

- Climate Change: *climate, change, global, world*.
- Feminis: *women, men, feminist, #gamergate*.
- Hillary Clinton: *hillary, clinton, vote, #wakeupamerica, #tcot*

No obstante, también hay tokens que se repiten a lo largo de todos los temas, como el mencionado anteriormente *user*, y, de nuevo, palabras temporales como *now, when, there*.

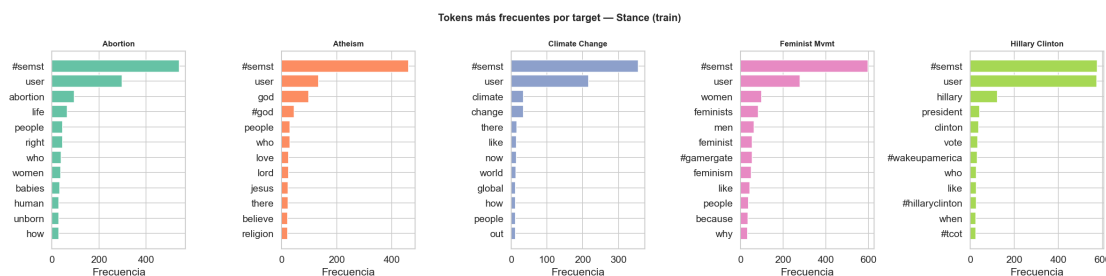


Figura 4.9: Top 12 tokens más frecuentes para los tweets de los distintos tópicos del dataset de stance.

Cabe destacar que el token más repetido en todos los tópicos es el *hashtag* #semst, que hace referencia a SemEval Stance Task (SemEval-2016 Task 6). Los creadores del dataset recolectaron tweets buscando #semst junto con términos del topic (*abortion, hillary, etc.*). Es un *hashtag* de colección metodológica pues los usuarios lo añadían para participar en debates cuyo contenido luego se anotó manualmente para el benchmark.

Para no confundir a los modelos de lenguaje y asegurarnos de que realmente usen el vocabulario específico para clasificar, debemos añadir semst al conjunto de *stopwords* antes de tokenizar o entrenar.

4.3. Procesamiento de los datos

Siguiendo la referencia de la Imagen 4.1, donde se muestra todo el flujo completo de los datos usados para este proyecto, desde que los obtenemos hasta que sacamos conclusiones relevantes, entendemos que el procesamiento de los datos se ha llevado a cabo haciendo uso de un cuaderno de Jupyter que carga distintos módulos Python. La idea es que el mismo notebook haga simultáneamente la normalización de los datos, la deduplicación por id y el split en conjuntos de entrenamiento, validación y test para las distintas fuentes. A continuación, profundizaremos un poco en cada una.

La normalización adoptada consistente en:

- Sustitución de menciones de usuario (@usuario) por el token especial @USER.
- Sustitución de URLs (<http://>, <https://> y t.co/) por el token especial HTTPURL.

- Decodificación de entidades HTML residuales (`&` → `&`, etc.).
- Colapso de espacios múltiples.
- Conservación de *hashtags*, emojis, mayúsculas y puntuación, dado que son señales semánticamente relevantes para el análisis de sentimiento y postura.
- Para los datos de la tarea de *stance*, se eliminó asimismo el *hashtag* `#semst`, que ya mencionamos en otra sección anterior y que hace referencia a la recolección del benchmark de 2016.

4.3.1. TweetEval

Como ya mencionamos cuando realizamos el EDA de los datos de TweetEval, para la tarea de sentimiento el conjunto de entrenamiento presentaba una distribución ligeramente desbalanceada y el conjunto de validación original era muy pequeño en comparación con el primero (2000 vs 45615), por lo que, para corregirlo hemos decidido aplicar una re-partición estratificada, que consiste en combinar los conjuntos de entrenamiento y validación originales y redistribuirlos en un split 90-10 manteniendo la distribución de clases positiva negativa y neutral.

El resultado es un nuevo conjunto de entrenamiento de 42.853 tweets y uno de validación de 4.762, ambos con distribución idéntica de clases (neutral 45,2 %, positiva 39,2 % y negativa 15,6 %). Por su lado, el conjunto de test original permanece intacto para poder obtener conclusiones válidas al comparar posteriormente los resultados de las métricas con los del estado del arte. El formato final de los splits es Parquet, y como ya hemos dicho, se almacenan en el bucket de GCS, con ruta `tweeteval/sentiment/{train,validation,test}.parquet`.

Por otra parte, en cuanto a la tarea de *stance*, para los cinco subtópicos de *stance* de TweetEval se aplica el mismo pipeline de normalización (con eliminación adicional de `#semst`), pero ahora a diferencia del caso del sentimiento, los splits originales de estos subtópicos se conservan sin re-partición porque sus conjuntos de validación ya son muy pequeños (entre 40 y 69 muestras), y combinarlos con el entrenamiento apenas añadiría capacidad de generalización. La Tabla 4.2 recoge el tamaño de cada split tras el procesamiento.

Tabla 4.2: Tamaño de los splits de TweetEval *stance* tras el procesamiento.

Subtópico	Train	Validación	Test
<code>stance_abortion</code>	587	66	280
<code>stance_atheism</code>	461	52	220
<code>stance_climate</code>	355	40	169
<code>stance_feminist</code>	597	67	285
<code>stance_hillary</code>	620	69	295
Total	2.620	294	1.249

Cada subtópico se almacena de forma independiente bajo `tweeteval/stance_{topico}/{split}.parquet` en el bucket de procesamiento de GCS.

4.3.2. Datos Scrapeados

El corpus scrapeado presenta un caso adicional que requiere tratamiento previo a la normalización que es la existencia de tweets duplicados, y es especialmente pronunciado en la campaña de España 2023, donde parte del *scraping* tuvo que repetirse tras un fallo de conectividad. La deduplicación se realiza por identificador único de tweet (`tweet.id`), criterio que garantiza la unicidad sin riesgo de eliminar tweets distintos con contenido idéntico.

4.3.3. Datos de Kaggle

Los tweets se cargan desde el fichero CSV y se someten al mismo pipeline de normalización y después se filtra por el idioma propicio para cada caso (inglés o español). Este corpus se almacena bajo `candidate_tweets/{campaña}.parquet`.

Como ya hemos mencionado, todos los datos procesados se serializan en formato Apache Parquet y se depositan en el bucket `gs://tfm-twitter-processed/` de Google Cloud Storage. Parquet fue elegido frente a CSV por tres razones: tipado fuerte de columnas (relevante para los identificadores numéricos de tweet que en CSV pueden ser truncados por limitaciones de precisión flotante), compresión columnar que reduce el tamaño en disco en torno a un 60 % respecto al CSV equivalente, y carga selectiva de columnas que acelera los notebooks de modelado al no tener que deserializar el fichero completo.

Adicionalmente, nos dimos cuenta de que los datos de Kaggle y los scrapeados presentaban una heterogeneidad de esquema no prevista inicialmente, lo que significa que, en el notebook de preprocesamiento, tuvimos que añadir algunas normalizaciones de esquema, para combinar los registros de ambas fuentes en uno único, como por ejemplo, el nombre del campo de texto principal (`text` en el scraper frente a `rawContent` en los ficheros Kaggle) u otras nomenclaturas como `retweetCount` frente a `retweet_count`.

Asimismo, se normalizaron las marcas temporales a `datetime64[UTC]`.

Para finalizar esta sección, mostramos ejemplos reales de los tweets procesados y que se encuentran en el bucket *processed* de GCS.

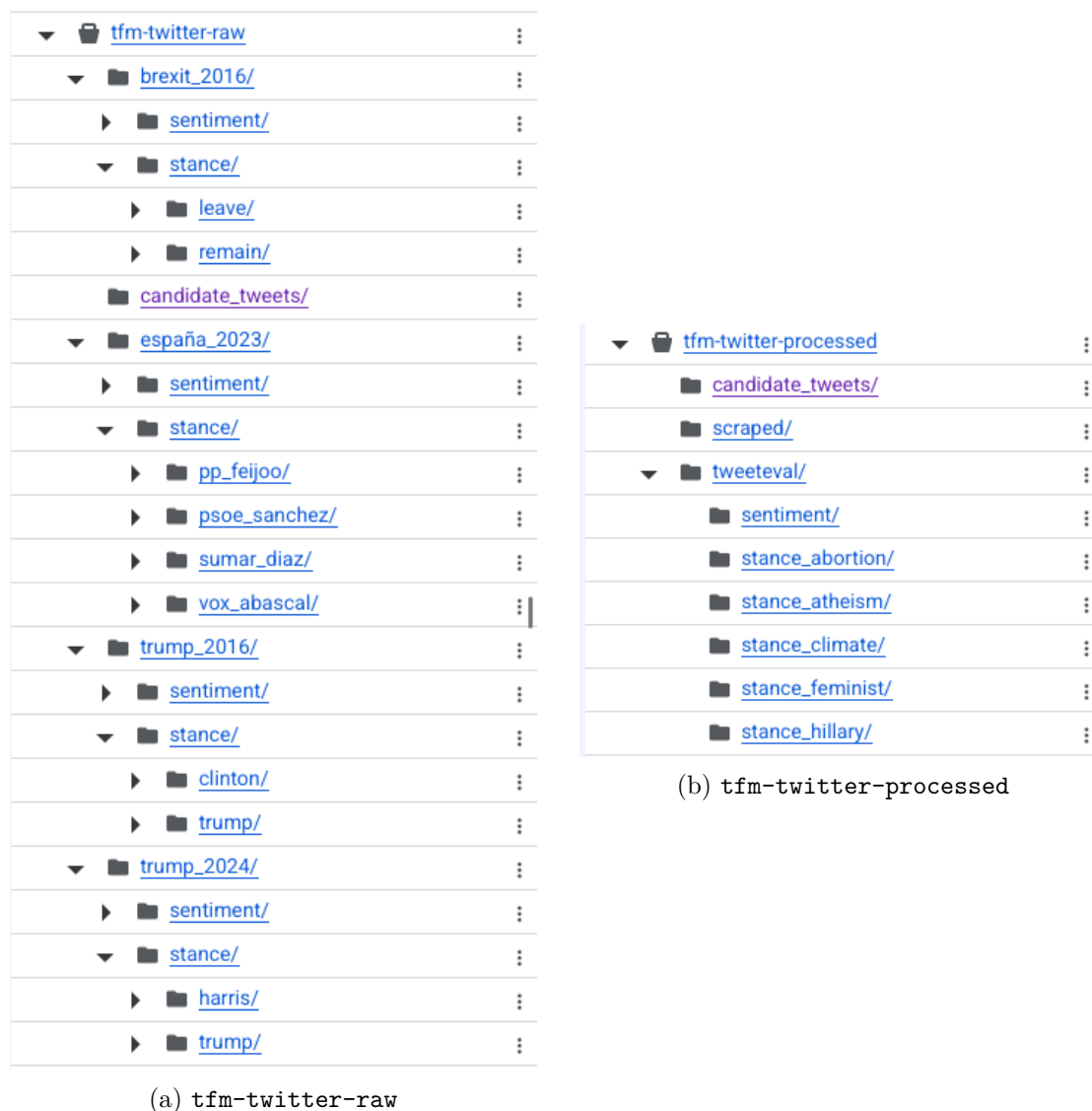


Figura 4.10: Contenido de los buckets de Google Cloud Storage tras el procesamiento completo. El bucket *raw* almacena los datos crudos en formato JSON, organizados por campaña, tarea y target. El bucket *processed* contiene los datos normalizados en formato Parquet, listos para su uso en los notebooks de modelado.

Campaña	Eje	Texto	BERTweet
brexit_2016	Pro-Leave	“WHAT WILL JEREMY CORBYN BE VOTING FOR? #VoteLeave #LeaveEU #Brexit @jeremy-corbyn https://t.co/k6Qm...”	@USER, HTTPURL
	Pro-Remain	“Gibraltar rock-solid in commitment to British membership of EU @GibStrongerIN #UKinEU #StrongerIn #Remain https://t.co/...”	@USER, HTTPURL
	Inmigración	“Adam Boulton points out Theresa May USELESS at immigration – got totally out of hand on her watch. @borisjohnson #voteleave”	@USER
	Económico	“BBC News – Five models for post-Brexit UK trade https://t.co/Xg2U84vGNr”	HTTPURL
trump_2016	Pro-Trump	“THE AMERICAN PEOPLE WILL NOW COME FIRST! #TRUMPENCE16”	sin cambio
	Pro-Clinton	“Lean back. #imwithher #hillary2016 #vote @ United States https://t.co/o2wg...”	HTTPURL
	Anti-Clinton	“Jerrold Nadler: James Comey ‘may very well have’ cost Hillary Clinton the election #JamesComey #HillaryEmails #FBI”	HTTPURL (×2)
	Electoral	“@Khanoisseur I recall an article on how 3rd party votes could also tip a swing state. Enter Pennsylvania”	@USER
trump_2024	Anti-MAGA	“@SenJoniErnst Why do you lie? She is not raising taxes on hard working Americans. You violate your oath everyday you support MAGA. #VoteHarris”	@USER
	Movilización	“All you MFs are LOUD AF when it comes to complaining but you ninjas ELECT not to vote. How about you ‘care’ yo ass down to those polls #vote”	sin cambio
	Post-elección	“President Trump should: 1. Fire them ALL. There are not enough good bureaucrats to spend time figuring out who they are. Start over.”	sin cambio
	Fact-checking	“Throughout Election Day, CBS News’ Confirmed team fact-checked reports of voter fraud, election hacking, and more.”	sin cambio
españa_2023	Anti-PSOE	“Venga Sevillanos, 25 años para tres trozos de la SE40 y una línea de metro, y ahora a pagar para salir de Sevilla por autovía. Seguid votando PSOE.”	sin cambio
	Pro-PSOE	“En el Mercadillo de Arcos con compañeros @PSOE_ARCOS explicando el proyecto del #PSOE para generar oportunidades.”	@USER
	Pro-PP	“Votar al @ppopular es la garantía de un Gobierno sin bloqueos y sin coaliciones. #EsElMomento #VotaPP”	@USER, HTTPURL
	Resultado	“Don Núñez sale tocadito... #23J”	sin cambio

Tabla 4.3: Ejemplos representativos extraídos de los parquets procesados por campaña. La columna *BERTweet* muestra la normalización aplicada: @usuario→@USER, URL→HTTPURL. Los textos se muestran truncados a 150 caracteres. Normalización basada en [Nguyen et al., 2020]

4.4. Entrenamiento y modelado

4.4.1. Análisis de sentimiento

Para el entrenamiento y modelado de los datos de TweetEval hemos empleado un notebook (ver Imagen 4.1) que hemos subido a Google Colaboratory con tal de poder usar la GPU T4 gratuita.

No obstante, debemos mencionar que, dado que el uso de GPU gratuita en Google Colaboratory es de aproximadamente 5 horas por sesión, hemos implementado un mecanismo de persistencia de los resultados, de forma que, los resultados escalares correspondientes tras cada ejecución completa (modelo con semilla determinada) se serializan en un fichero JSON y se almacenan en el bucket *curated* de GCS (`gs://tfm-twitter-curated/results/sentiment_grid_cache.json`). Al iniciar una nueva sesión, el cuaderno carga automáticamente este registro y omite las combinaciones ya evaluadas, reanudando el entrenamiento desde el punto de interrupción. Este esquema permite distribuir la ejecución del grid en múltiples sesiones sin pérdida de reproducibilidad, ya que cada resultado queda vinculado a su combinación exacta de modelo, tasa de aprendizaje y semilla de inicialización.

También hemos subido el archivo *service-account.json* que contiene las credenciales necesarias para que se pueda acceder directamente a los buckets de GCS y hacer uso así de los datos allí procesados y reparticionados, tal y como mencionamos en la anterior sección.

Antes de nada, me gustaría comentar que, lo que hemos llevado a cabo no es un entrenamiento desde cero como tal, sino que hemos partido de distintos modelos pre-entrenados y los hemos adaptado mediante ajuste fino o *fine-tuning* a nuestras tareas concretas de interés, en este caso, la clasificación de sentimiento. Esto ya lo mencionamos en la sección 3.1.3.

Los modelos *fine tuneados* son los ya vistos en la Tabla 2.3, enumerados de 0 a 5. Estas 6 configuraciones se pueden agrupar en 3 categorías fundamentales, y se resumen en la siguiente tabla recordatoria.

El grupo 1 es el de dominio en Twitter en inglés (E0, E1 y E2). El grupo 2 es la línea base genérica (E5) y el grupo 3 es el de cruce lingüístico (E3 y E4). E5 actúa como base genérica porque el único factor que cambia respecto a E1 o E2 es el corpus de entrenamiento (E5 nunca vio tweets en la primera fase). Los corpus se pueden ver en la Tabla 3.1. Así, si luego vemos que E1 y E2 superan a E5 en F1-Macro será debido al pre-entrenamiento en dominio de Twitter, mientras que si la diferencia es mínima, entonces significará que el dominio importa poco.

Para cada modelo, no obstante, se usa una estrategia distinta. Para los modelos que ya incorporan una cabeza de clasificación de sentimiento ajustada por sus autores originales (E0 y E3), se adopta en cambio una estrategia de evaluación *zero-shot*, es decir, sin ningún entrenamiento adicional sobre nuestros datos, por las siguientes razones. E0 ya fue *fine tuneado* en [Barbieri et al., 2020] sobre el propio conjunto de TweetEval, y someterlo a un ajuste fino adicional sobre los mismos datos violaría la independencia del conjunto de test, sesgando artificialmente sus resultados. Por su lado, E3 fue igualmente ajustado

ID	Modelo (HuggingFace)	Estrategia	Ref. benchmark
E0	twitter-roberta-base-sentiment	ZS	TweetEval
E1	twitter-roberta-base	FT	TweetEval
E2	bertweet-base	FT	TweetEval
E3	twitter-xlm-roberta-base-sentiment	ZS	UMSAB-EN
E4	robertuito-sentiment-analysis	ZS	TASS 2020*
E5	bert-base-uncased	FT	TweetEval

*Benchmark en castellano, resultados no comparables directamente con TweetEval.

Tabla 4.4: Banco de modelos evaluados en la tarea de sentimiento. ZS = *zero-shot*; FT = *fine tuning*.

sobre UMSAB-EN [Barbieri et al., 2022], que es el mismo corpus SemEval-2017 que subyace a TweetEval por lo que, mismo argumento. Por su parte, E4 fue pre-entrenado sobre tweets en castellano y ajustado sobre TASS 2020 [Pérez et al., 2022]; se evalúa en modalidad *zero-shot* para medir su capacidad de transferencia cruzada entre idiomas sin adaptación adicional, lo que constituye por sí mismo un experimento de análisis lingüístico.

Estos modelos en modalidad *zero-shot* actúan como cota de referencia pues si el ajuste fino de E1, E2 o E5 supera a E0 en el mismo dominio, indica que la adaptación con datos etiquetados proporciona valor adicional más allá del pre-entrenamiento genérico.

El *fine tuning* de E1, E2 y E5 se realiza mediante la librería Transformers de Hugging Face. Y para compensar el desequilibrio de clases, usamos una función de pérdida ponderada, ya mencionada en 3.1.3, donde los pesos resultantes para cada clase son aproximadamente: $w_{\text{neg}} \approx 2,13$, $w_{\text{neu}} \approx 0,73$ y $w_{\text{pos}} \approx 0,85$.

Además, hemos usado distintas semillas de inicialización en el *grid search*, para poder asegurar la reproducibilidad de los resultados, lo que supone en 9 ejecuciones independientes por modelo.

$$\eta \in \{10^{-5}, 2 \times 10^{-5}, 5 \times 10^{-5}\}, \quad s \in \{42, 123, 2024\} \quad (4.1)$$

Por otro lado, los hiperparámetros fijos son: longitud máxima de secuencia de 128 tokens, tamaño de lote de 32, 5 épocas y regularización L_2 con $\lambda = 0.01$. Se activa parada anticipada (*early stopping*) basada en el F1-Macro sobre el conjunto de validación, cargando automáticamente los pesos del mejor punto de control al finalizar cada ejecución.

La combinación (η^*, s^*) que maximiza el F1-Macro en validación se selecciona como configuración óptima. Con dicha configuración, el rendimiento final se reporta como la media y la desviación típica del F1-Macro sobre el conjunto de test evaluado bajo las tres semillas iniciales.

$$\overline{F_1} \pm \sigma = \frac{1}{|S|} \sum_{s \in S} F_1^{(s)} \pm \sqrt{\frac{1}{|S|} \sum_{s \in S} (F_1^{(s)} - \overline{F_1})^2} \quad (4.2)$$

4.4.2. Detección de postura

Para la tarea de detección de postura hemos empleado un subconjunto de los modelos empleados en la tarea de análisis de sentimiento, aplicados ahora sobre los datos de TweetEval para stance.

De nuevo, estos 4 modelos, que se presentan en la siguiente tabla, son ejemplos de arquitecturas basadas en Transformers, ya que el estado del arte se rige por estos para alcanzar los mejores resultados.

Debemos mencionar que, para la detección de postura, es necesario realizar un *fine tuning* independiente para cada uno de los 5 tópicos del dataset, dado que la postura es dependiente del target.

ID	Modelo	Estrategia	Ref. benchmark
S0	twitter-roberta-base	ZS	TweetEval [Barbieri et al., 2020]
S1	twitter-roberta-base	FT $\times 5$	TweetEval [Barbieri et al., 2020]
S2	bertweet-base	FT $\times 5$	TweetEval [Nguyen et al., 2020]
S3	twitter-xlm-roberta-base-sentiment	ZS	UMSAB-EN [Barbieri et al., 2022]

Tabla 4.5: Estrategias de modelado para *Stance Detection*. ZS = *zero-shot*; FT = *fine tuning*.

También, hay que mencionar que ahora ya no se realiza búsqueda de hiperparámetros por grid search debido a que el dataset de stance es mucho más pequeño y esta tarea la consideramos complementaria en el trabajo, por lo que vamos a entrenar con una única *learning rate* de $\eta = 2 \times 10^{-5}$ durante cinco épocas y una única semilla de inicialización *seed=42*.

La tarea de clasificación de postura presenta condiciones más restrictivas que la tarea de sentimiento, lo cual nos obliga a adoptar estrategias un poco diferentes. En particular, como ya vimos en el análisis descriptivo de los datos, los conjuntos de validación de TweetEval para stance tienen entre 40 y 69 muestras por tópico, lo cual es muy poco, es insuficiente para poder realizar un *early stopping* de los modelos estadísticamente fiable. Por ello, hemos decidido mezclar los splits originales de entrenamiento y validación para maximizar los datos de entrenamiento disponibles. Esto nos obliga a realizar la evaluación en el conjunto de test. No es lo ideal pero sí creo que lo más coherente. Hemos decidido escoger esta técnica de combinación de los splits frente a otras como la validación cruzada porque esta última sería más útil si fuéramos a seleccionar el valor de la tasa de aprendizaje, pero como ya tomamos $\eta = 2 \times 10^{-5}$ directamente (valor óptimo en el análisis de sentimiento sobre el mismo tipo de arquitectura), nos conviene más combinar train y val, lo cual a su vez acelera el entrenamiento, porque tendríamos muchos folds para a su vez, muchos tópicos. En particular, $k \times 5 \times 2$ ejecuciones de

fine tuning, lo que supone un coste computacional desproporcionado para una tarea de carácter más bien complementario.

Asimismo, cabe mencionar que modelos específicamente orientados al español como el robertuito, que sí hemos usado para sentimiento, se preservan para etapas posteriores del trabajo para la tarea de stance.

Por otra parte, para el entrenamiento de stance también vamos a usar una función de pérdida ponderada. No obstante; a diferencia del análisis de sentimiento, donde teníamos un único vector de pesos para todo el corpus, ahora los pesos se calculan de forma independiente para cada subtópico, ya que la distribución entre las clases *none*, *favor* y *against* varía notablemente entre ellos.

Los pesos se obtienen mediante el criterio de clases balanceadas aplicado sobre el conjunto combinado train + val para cada subtópico, de forma que tenemos:

Subtópico	N	w_{none}	w_{against}	w_{favor}
abortion	653	1.230	0.613	1.799
atheism	513	1.462	0.563	1.859
climate	395	0.784	8.778	0.621
feminist	664	1.757	0.675	1.054
hillary	689	1.290	0.584	1.946

Tabla 4.6: Pesos de clase por subtópico (función de pérdida ponderada, train+val).

4.5. Inferencia y aplicación de los modelos sobre los datos reales.

Como ya vimos en el esquema 4.1, tras el *fine tuning* y evaluación de los modelos correspondientes, y tras seleccionar aquellos que mejor performan sobre TweetEval en test, es hora de aplicarlos sobre datos reales, nuestros datos scrapeados directamente desde Twitter y los datos de Kaggle que previamente ya pasaron por el preprocesamiento de la sección 4.3 y normalización BERTweet.

Con estos datos de las 4 campañas políticas de interés vamos a realizar tanto una inferencia de sentimiento, mediante los modelos E0 (cardiffnlp/twitter-roberta-base-sentiment), aplicado como *off-the shelf* (sin *fine tuning* adicional sobre nuestros datos) y E4 (pysentimiento/robertuito-sentiment-analysis); como una inferencia de postura a través del modelo S1, cargado desde GCS (gs://tfm-twitter-curated/models/stance/{topic}). El modelo en español E4 se va a aplicar únicamente sobre los datos relativos a la campaña España 2023, debido a que, como ya sabemos, este modelo está pre-entrenado en español y nos serviría para hacer un análisis propio en español, además del análisis correspondiente con E0 para poder tener comparabilidad *cross-campaign*, que es lo que más nos interesa.

No obstante, esto no lo podemos replicar en el caso de *stance* ya que no tenemos ningún

modelo de postura equivalente en español, por tanto, esto es una limitación explícita y solo podremos aplicar S1, que entendemos que funcionará peor sobre España 2023.

La elección de E0 como modelo principal se justifica por ser el mejor modelo en inglés de los evaluados (F1-Macro=72.36 % sobre TweetEval), lo cual se puede ver con más detalle en la sección de Análisis de resultados. E4 se incorpora exclusivamente para la campaña española al estar pre-entrenado sobre Twitter en español. Su aplicación paralela a E0 sobre los mismos datos permite cuantificar el impacto del desajuste lingüístico, pero, de nuevo, recordamos que sus resultados sobre TASS 2020 no son directamente comparables con TweetEval. Para postura se emplea S1 por ser el modelo en obtener un mejor F1-Average en evaluación, lo cual también se puede ver con más detalle en Análisis de resultados.

Así, en el notebook `03_sentiment_scraped.ipynb` tendremos un pipeline Hugging-Face en batch, donde el output será la etiqueta (negativo, positivo, neutro) junto a un score de confianza por tweet.

Y en el notebook `05_stance_scraped.ipynb` tendremos una selección de pares campaña-tópico (porque no todos los tópicos de SemEval son relevantes para todas las campañas electorales reales), que identificará la relevancia política documentada, y, posteriormente, un pipeline por tópico, donde el output será la etiqueta (*favor*, *against*, *none*), junto con un análisis cruzado de sentimiento y postura que nos va a permitir detectar si ciertos grupos ideológicos exhiben ciertos patrones de negatividad diferenciados, clave para poder sacar conclusiones reales sobre cómo actúan ciertos grupos defensores de posturas político-sociales a través de las redes sociales.

Los pares campaña-tópico que hemos decidido usar son los que se muestran en la Tabla ?? a continuación:

En ambos notebooks hemos incorporado un diccionario de eventos relevantes por campaña, definidos manualmente a partir de literatura política. Su función es, por un lado, servir como líneas verticales en los timelines de sentimiento y postura para las campañas, lo que nos va a permitir inspeccionar visualmente si un evento particular produce un cambio de tendencia en el discurso; y, por otro lado, cuantificar el impacto de dichos eventos.

Los eventos seleccionados por campaña son:

Tabla 4.7: Pares campaña-tópico seleccionados para el análisis de postura

Campaña	Tópico	Justificación
trump_2016	hillary	Hillary Clinton fue la candidata directa opuesta a Trump; el tópico SemEval mide explícitamente la postura hacia Clinton, el par más natural y políticamente directo del conjunto.
trump_2024	abortion	La sentencia <i>Dobbs v. Jackson</i> (2022) derogó <i>Roe v. Wade</i> , convirtiendo el aborto en el principal eje de movilización de la campaña 2024.
	feminist	La candidatura de Kamala Harris elevó el debate de género al centro de la agenda; Trump generó controversia continuada con declaraciones sobre el rol de la mujer.
	climate	Trump anunció en campaña la salida del Acuerdo de París y el desmantelamiento de la EPA, convirtiendo el clima en un eje de fractura ideológica explícito.
españa_2023	feminist	La Ley del Sí es Sí dominó el debate preelectoral; PP y Vox articularon un discurso crítico con el feminismo, mientras PSOE y Sumar lo reivindicaban.
	abortion	El aborto formó parte de la agenda con posiciones antagónicas: PSOE/Sumar defendían la ampliación de derechos reproductivos; PP y Vox planteaban su restricción.
	climate	Las sequías históricas del verano de 2023 y la agenda verde de PSOE/Sumar contrastaron con el negacionismo climático explícito de Vox.
brexit_2016	climate	El Brexit implicó la salida del marco de política medioambiental europeo; el tópico permite detectar si los tweeteros pro-Leave y pro-Remain diferían en su postura climática, una fractura ideológica documentada en la literatura.

Campaña	Fecha	Evento
brexit_2016	16 may 2016	Informe HM Treasury (impacto económico Brexit)
	16 jun 2016	Asesinato de Jo Cox
	23 jun 2016	Día del referéndum
	24 jun 2016	Leave gana — dimisión de Cameron
trump_2016	26 sep 2016	1 ^{er} debate Trump-Clinton
	07 oct 2016	Grabación <i>Access Hollywood</i>
	09 oct 2016	2 ^o debate
	28 oct 2016	Carta Comey/FBI (emails Hillary)
	08 nov 2016	Día de elecciones
	09 nov 2016	Victoria de Trump
trump_2024	10 sep 2024	Debate Trump-Harris
	05 oct 2024	Huracán Milton
	05 nov 2024	Día de elecciones
	06 nov 2024	Victoria de Trump
españa_2023	10 jul 2023	Debate Feijóo-Sánchez (AtresMedia)
	20 jul 2023	Último día de campaña
	23 jul 2023	Elecciones 23J
	24 jul 2023	PP gana sin mayoría — bloqueo de gobierno

Tabla 4.8: Eventos que hemos considerado clave anotados por campaña electoral

5 Análisis de resultados

5.1. Análisis de sentimiento

En primer lugar, mostramos los resultados completos por modelo, semilla y tasa de aprendizaje.

ID	Modelo	F1 por clase (%)				σ	Acc (%)	LR
		Neg	Neu	Pos	Macro			
E0	twitter-roberta-base-sentiment	74.50	71.63	70.95	72,36	—	72.48	—
E1	twitter-roberta-base	75.01	68.94	71.12	71.69	0.19	71.52	2e-5
E2	bertweet-base	75.04	68.94	71.05	71.68	0.47	71.53	2e-5
E3	twitter-xlm-roberta-base-sentiment	71.89	63.71	68.53	68.04	—	67.87	—
E5	bert-base-uncased	72.49	65.64	68.20	68.78	0.13	68.64	1e-5
E4	robertuito-sentiment-analysis	69.56	64.94	61.21	65.24	—	66.03	—
Referencia TweetEval [Barbieri et al., 2020]		—	—	—	72,80	—	—	—

Tabla 5.1: Resultados de los seis modelos evaluados en el test set de TweetEval Sentiment (n=12,284). Los modelos E1, E2 y E5 reportan el F1-Macro del mejor run (LR óptima, semilla óptima); la columna σ indica la desviación estándar sobre las tres semillas con esa LR. La referencia del paper es Barbieri et al. [2020] (F1-Macro=72,8 %).

Modelo	LR	seed 42	seed 123	seed 2024	$\bar{x}$	σ
E1 twitter-roberta-base	1e-5	70,58	70,82	71,10	70,83	0,21
	2e-5	71,11	71,12	71,52	71,25	0,19
	5e-5	70,41	70,51	71,14	70,69	0,32
E2 bertweet-base	1e-5	70,95	70,81	70,59	70,78	0,15
	2e-5	71,81	70,96	70,70	71,16	0,47
	5e-5	70,83	71,36	70,60	70,93	0,32
E5 bert-base-uncased	1e-5	68,72	68,85	68,54	68,70	0,13
	2e-5	68,26	68,55	68,38	68,40	0,12
	5e-5	68,47	68,09	67,97	68,18	0,21

Tabla 5.2: F1-Macro (%) en validación para cada combinación de tasa de aprendizaje y semilla aleatoria. En negrita la configuración seleccionada por modelo. $\bar{x}$ y σ calculados sobre las tres semillas.

ID	Modelo	F1-Neg	F1-Neu	F1-Pos	F1-Macro
E2	bertweet-base	75.04	68.94	71.05	71.68
E1	twitter-roberta-base	75.01	68.94	71.12	71.69
E0	twitter-roberta-base-sentiment	74.50	71.63	70.95	72.36*
E5	bert-base-uncased	72.49	65.64	68.20	68.78
E3	twitter-xlm-roberta-base-sentiment	71.89	63.71	68.53	68.04
E4	robertuito-sentiment-analysis	69.56	64.94	61.21	65.24

* E0 sin fine-tuning adicional; F1-Macro global más alto pese a F1-Neg inferior a E1/E2.

Tabla 5.3: F1 por clase (%) en el test set de TweetEval Sentiment para cada modelo (mejor configuración). La distribución de clases en test es: negativo 32,3 %, neutro 48,3 %, positivo 19,3 %.

Si nos fijamos en 5.2, vemos que el modelo E0 ALCANZA UN f1-Macro del 72.36 % en el conjunto de test de TweetEval, superando así a todos los modelos *fine tuneados*. E1, por su lado, obtiene 71.68 ± 0.19 ; y E2, 71.68 ± 0.47 . Esto significa que la diferencia entre E0 y el mejor modelo *fine tuneado* es de apenas 0.67 ppuntos porcentuales, por lo que el margen de mejora es muy escaso cuando el modelo ya fue pre-entrenado específicamente sobre ese benchmark.

Recordemos que el modelo de referencia del paper original [Barbieri et al., 2020] reportó un F1-Macro de 72.8 %, quedando solo un poco por encima de E0 y dentro del margen de variabilidad entre semillas.

Si nos fijamos, además, el ranking de los modelos está dominado por la especificidad del par dominio-tarea. Por eso, E0, que está pre-entrenado con Twitter, ocupa el primer puesto, y E1 y E2, cuyos encoders fueron pre-entrenados sobre corpus de Twitter, aunque no presenten una cabeza de sentimiento pre-entrenada en TweetEval, quedan solo 0.7 puntos porcentuales por debajo. E3, por su lado, que es el modelo XLM-RoBERTa multilingüe cae hasta 68.04 %, quizá porque su oreentrenamiento multilingüe diluye la especialización en inglés coloquial. E5 se queda en 68.78 %, lo que nos confirma que el corpus de pre-entrenamiento sí que es importante, porque como mencionamos en la sección de modelado, E5 está pre-entrenado en la Wikipedia y el BookCorpus, pero en ninguno de esos dos registros hay un uso coloquial de abreviaturas, *hashtags*, emojis... Y, finalmente, E4 (RoBERTuito, modelo español) obtiene el peor resultado (65,24 %), esperado dado que TweetEval está en inglés.

Sobre la estabilidad de los modelos, podemos mirar la tabla 5.1, que nos muestra los resultados del *grid search* y ver que el modelo E1 es el más estable pues presenta un $std = 0.19pp$, mientras que E2 es el más inestable, pues es el que presenta una mayor varianza.

Por último, si tenemos que hablar de la capacidad de diferenciar las clases de los modelos, y nos fijamos en 5.3, podemos decir que los 3 modelos *fine tuneados* presentan el mismo patrón $F1_{negativo} > F1_{positivo} > F1_{neutro}$. Como ya vimos, el conjunto de test presenta un desequilibrio inverso al de train, y, al haber usado una función de pesos ponderada, la clase negativa, pese a estra infrarrepresentada en el entrenamiento, resulta

ser la mejor predicha en el test. En todos los casos, la clase neutra sigue siendo la más difícil de capturar. E0 es el único modelo que supera el 70 % para la clase neutra.

Como E0 resultó ser el mejor modelo de los 5 probados, vamos a añadir la matriz de confusión correspondiente, que nos va a mostrar dónde falla más, la distribución de confianza del modelo, y el t-SNE del mismo.

	Pred. negativo	Pred. neutro	Pred. positivo
Real negativo (n=3972)	3102	817	53
Real neutro (n=5937)	1200	4135	602
Real positivo (n=2375)	53	656	1666

Tabla 5.4: Matriz de confusión de E0 (`twitter-roberta-base-sentiment`, sin fine-tuning adicional) sobre el test set de TweetEval Sentiment (n=12284). Filas = clase real; columnas = clase predicha.

Si nos fijamos en 5.4, las clases positiva y negativa casi nunca se confunden pues E0 solo presenta 53 de estos casos. Las que más se confunden son la clase neutra como negativa (1200 casos) y la clase negativa como neutra (817 casos).

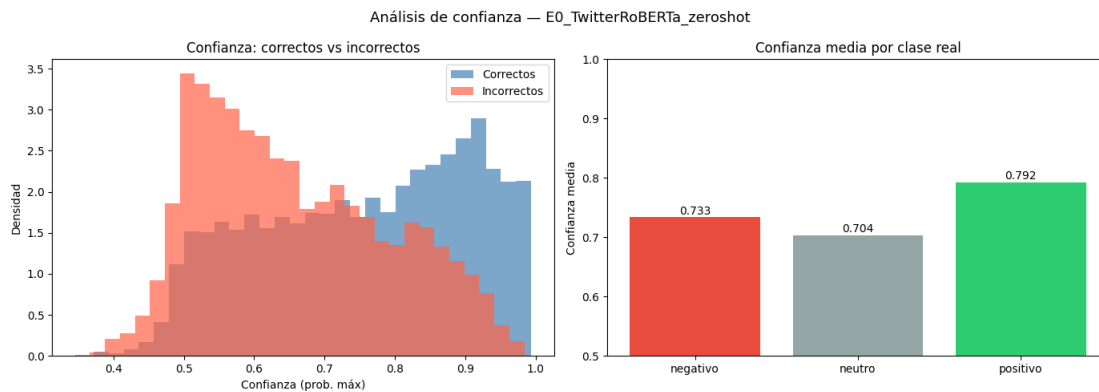


Figura 5.1: Distribución de confianza de las clasificaciones correctas frente a las incorrectas y confianza media por clase.

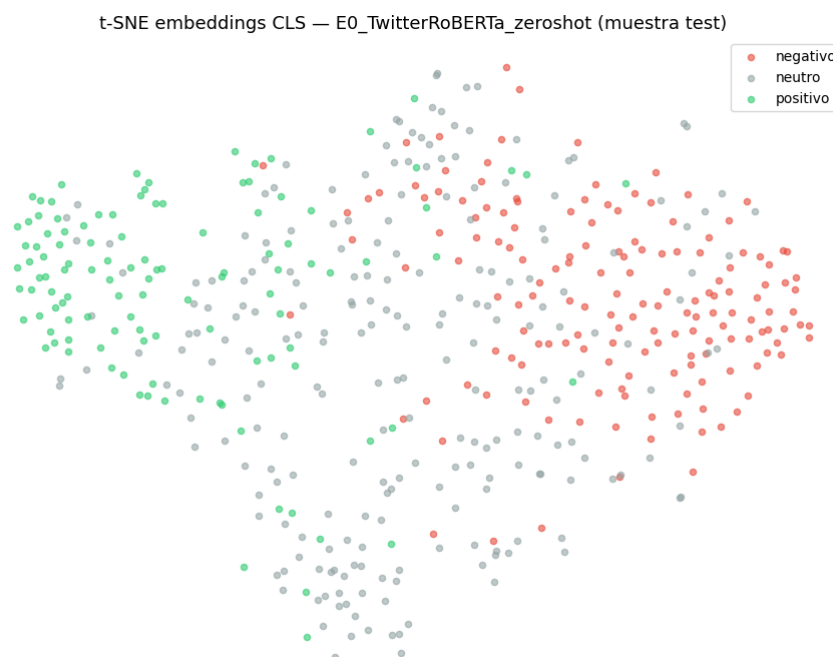


Figura 5.2: Proyección t-SNE de los embeddings del modelo E0 sobre el test set (n=1284). Perplexity= 30, 1000 iteraciones.

El t-SNE (*t-distributed Stochastic Neighbor Embedding*) es una técnica no supervisada de reducción no lineal de la dimensionalidad para la exploración de datos y la visualización de datos de alta dimensión. El algoritmo intenta mapear puntos de datos de mayor dimensión en un espacio de menor dimensión y preserva las similitudes entre pares. Se consigue minimizando la divergencia entre la distribución de probabilidad de la alta dimensión original y la de la dimensión inferior. Para ello, el algoritmo utiliza el descenso gradiente para minimizar la divergencia. [Awan, 2024].

Esto, lo que nos permite es ver la separabilidad entre clases, pues si los 3 grupos formaban clusters compactos y bien separados, significa que el modelo aprendió bien representaciones discriminativas. Se trata de una técnica que hemos usado mucho en la asignatura de *Machine Learning*. En nuestro caso, la Figura 5.2 muestra que los grupos negativo y positivo presentan núcleos compactos con escaso solapamiento entre sí, lo que indica que el modelo aprendió representaciones que separan eficazmente los dos lados del espectro emocional. La clase neutro, en cambio, ocupa una región difusa que se superpone parcialmente con ambos extremos, lo cual explica su menor F1. Los tweets neutros son lingüísticamente heterogéneos y esta estructura refleja la naturaleza intrínsecamente ambigua de la clase neutra en el contexto de Twitter, donde la frontera entre una opinión templada y la ausencia de opinión es frecuentemente difusa.

5.2. Detección de postura

A continuación, mostramos los resultados para el notebook de detección de postura. Comenzamos por la métrica F1-Average por estrategia y tópico.

Estrategia	abortion	atheism	climate	feminist	hillary	Media
S0 twitter-roberta zero-shot	0,00	19,32	5,56	3,55	3,76	6,44
S3 xlm-roberta zero-shot	20,44	25,68	17,60	26,32	34,77	24,96
S2 bertweet-base fine-tune	61,00	65,51	52,69	56,89	65,18	60,25
S1 twitter-roberta fine-tune	63,96	71,90	45,28	59,38	67,68	61,64
Referencia TweetEval [Barbieri et al., 2020]						69,30

Tabla 5.5: F1-Average (%) por estrategia y tópico en el test set de TweetEval Stance. $F1\text{-Average} = (F1_{\text{against}} + F1_{\text{favor}}) / 2$ (métrica oficial SemEval-2016, excluye *none*). Referencia TweetEval [Barbieri et al., 2020]: 69,3 %.

Tópico	Modelo	F1-none	F1-against	F1-favor	F1-Average	F1-Macro
abortion	S1	57,66	72,96	54,96	63,96	61,86
	S2	52,03	67,31	54,69	61,00	58,01
atheism	S1	72,73	83,33	60,47	71,90	72,18
	S2	68,42	80,43	50,60	65,51	66,48
climate	S1	76,71	0,00	90,55	45,28	55,75
	S2	72,46	16,67	88,72	52,69	59,28
feminist	S1	65,31	67,34	51,43	59,38	61,36
	S2	68,00	61,92	51,85	56,89	60,59
hillary	S1	63,95	79,55	55,81	67,68	66,44
	S2	58,82	76,75	53,61	65,18	63,06

Tabla 5.6: F1 por clase (%) de S1 y S2 (modelos fine-tuneados) por tópico. F1-Average es la métrica principal; F1-none se incluye solo como referencia.

Para los resultados de *stance*, como podemos apreciar mirando la Tabla 5.6, ahora existe un contraste radical entre los modelos *zero-shot*, como son S0 y S3 frente a los otros (S1 y S2). Los primeros obtienen un F1-average de apenas 6.44 % y 24.96 % respectivamente, frente al 61.64 % y 60.25 % respectivo de los segundos.

Esta caída en performance de los modelos *zero-shot* se explica porque la postura es una tarea muy dependiente del contexto político. El clasificar como rechazar o aceptar requiere un conocimiento sobre la posición política que los modelos de sentimiento general no poseen.

De hecho, si nos fijamos en la tabla, S0 alcanza un 0 % de F1-favor en cuatro de los cinco tópicos, lo que revela que el modelo interpreta términos políticamente cargados como negativos en términos de sentimiento pero no puede mapear esa polaridad a una postura concreta.

Entre S1 y S2, S1 supera a S2 en todos los tópicos excepto climate, con ventajas de entre 2 y 6 puntos porcentuales. En climate, que ya sabemos que es el tópico que presenta menso muestras y más ejemplos a favor.

Si hacemos un análisis por clase transversal, la clase favor presenta sistemáticamente el F1 más bajo en tópicos con una distribución equilibrada (abortion: 54-55 %; feminist: 51-52 %; hillary: 56 %), mientras que against es la más fácil de aprender (abortion: 67-73 %; atheism: 80-83 %; hillary: 76-80 %). Esto es coherente con el desequilibrio de clases en el corpus y con la expresión lingüística: el rechazo político tiende a formularse con lenguaje más explícito y homogéneo (‘‘I oppose’’, ‘‘I’m against’’, uso de insultos directos), mientras que el apoyo suele ser más implícito, contextual y lingüísticamente diverso.

No obstante, si comparamos con el modelo de referencia, la media de F1-macro de S1 (63.52 %) queda 5,8 pp por debajo de la referencia del paper TweetEval (69,3 %), aunque el gap se considera aceptable dado el contexto del trabajo y la limitación de los conjuntos de entrenamiento por tópico (395–689 muestras).

A continuación, adjuntamos una imagen donde se ve la comparativa del F1-Average por clase y por estrategia, y donde se aprecia de una manera más directa el contraste entre las estrategias *zero-shot* y *fine tuning*, que en el caso del análisis de sentimiento, no era tan brusca.

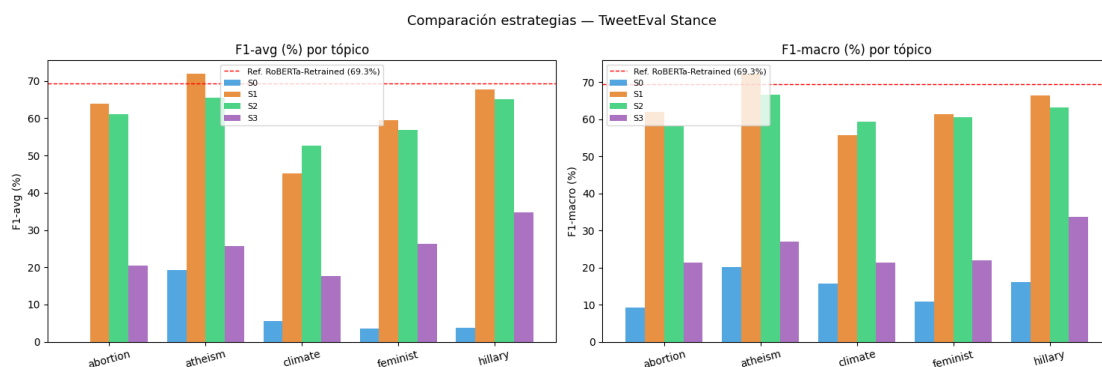


Figura 5.3: Comparación de las cuatro estrategias (S0–S3) por tópico en términos de F1-Average (izquierda) y F1-Macro (derecha). La línea roja discontinua indica la referencia del paper TweetEval [Barbieri et al., 2020] (F1-Average=69.3 %). El contraste entre los modelos *zero-shot* (S0, S3) y los *fine-tuneados* (S1, S2) evidencia que la detección de postura requiere adaptación supervisada por tópico: S0 colapsa a valores próximos a cero en cuatro de los cinco tópicos, mientras que S1 alcanza F1-Average>60 % en todos salvo *climate*, tópico estructuralmente desequilibrado.

Como S1 es el mejor modelo, adjuntamos su matriz de confusión:

Cabe destacar que en los temas de aborto y feminismo, las clases a favor en contra se confunden bilateralmente, lo que quizá implique que para estos temas en particular como son los temas sociales, el lenguaje de postura pueda sere menos explícito.

En el tópico del cambio climático, como ya hemos mencionado, no clasifica ninguna

Real \ Pred	abortion			atheism			climate			feminist			hillary		
	no	ag	fa	no	ag	fa	no	ag	fa	no	ag	fa	no	ag	fa
none	32	5	8	24	3	1	28	0	7	32	2	10	47	27	4
against	32	116	41	13	120	27	2	0	9	21	100	62	17	142	13
favor	2	8	36	1	5	26	8	0	115	1	12	45	5	16	24

Tabla 5.7: Matrices de confusión de S1 (`twitter-roberta-base` fine-tuneado) por tópico sobre el test set. Filas = clase real; columnas = clase predicha. Clases: *none* (0), *against* (1), *favor* (2).

muestra como *against*, y en el tópico Hillary, la clase *none* se confunde con *against*. Hay presencia de tweets ambiguos sobre Hillary que son interpretados como crítica.

5.3. Resultados de la inferencia sobre los datos reales.

5.3.1. Análisis de sentimiento.

En primer lugar, mostramos los resultados de los modelos sobre estos datos

5.3.2. Detección de postura.

5.4. Análisis integrado y multipartidismo.

6 Conclusiones

Bibliografía

- Association of Internet Researchers. Ethical decision-making and internet research: Recommendations from the AoIR ethics working committee (version 2.0). Technical report, Association of Internet Researchers, 2012. URL <https://aoir.org/reports/ethics2.pdf>.
- Abid Ali Awan. Introducción al t-sne, 2024. <https://www.datacamp.com/es/tutorial/introduction-t-sne>.
- Alexandra Balahur. Sentiment analysis in social media texts. In *Proceedings of the 4th workshop on computational approaches to subjectivity, sentiment and social media analysis*, pages 120–128, 2013.
- Francesco Barbieri, Jose Camacho-Collados, Leonardo Neves, and Luis Espinosa-Anke. Tweeteval: Unified benchmark and comparative evaluation for tweet classification. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1644–1650. Association for Computational Linguistics, 2020.
- Francesco Barbieri, Luis Espinosa Anke, and Jose Camacho-Collados. XLM-T: Multilingual language models in Twitter for sentiment analysis and beyond. In *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, pages 258–266, Marseille, France, 2022. European Language Resources Association. URL <https://aclanthology.org/2022.lrec-1.27>.
- Alessandro Bessi and Emilio Ferrara. Social bots distort the 2016 us presidential election online discussion. *First Monday*, 21(11), 2016. doi: 10.5210/fm.v21i11.7090.
- Alexandre Bovet and Hernán A Makse. Influence of fake news in twitter during the 2016 us presidential election. *Nature Communications*, 10(1):7, 2019. doi: 10.1038/s41467-018-07761-2.
- Alexis Conneau, Kartikay Karthikeyan, Guillaume Lample, et al. Unsupervised cross-lingual representation learning at scale. In *Proceedings of ACL*, 2020.
- DataCamp. ¿qué es el bert? introducción a los modelos bert, 2024. <https://www.datacamp.com/es/blog/what-is-bert-an-intro-to-bert-models>.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, volume 1, pages 4171–4186, 2019.

- Paul Ekman. An argument for basic emotions. *Cognition & Emotion*, 6(3-4):169–200, 1992.
- Andrea Esuli and Fabrizio Sebastiani. Sentiwordnet: A publicly available lexical resource for sentiment analysis and opinion mining. In *Proceedings of the 5th Conference on International Language Resources and Evaluation (LREC'06)*, pages 417–422, Génova, Italia, 2006. European Language Resources Association (ELRA).
- Anastasia Giachanou and Fabio Crestani. Like it or not: A survey of twitter sentiment analysis. *ACM Computing Surveys (CSUR)*, 49(2):1–42, 2016. doi: 10.1145/2938640.
- Yuriy Gorodnichenko, Tho Pham, and Oleksandr Talavera. Social media, sentiment and public opinions: Evidence from twitter and surveys. *Journal of Monetary Economics*, 118:236–254, 2021.
- Christos Hadjinikolis, 2020. URL <https://www.kaggle.com/datasets/chadjinik/labelledbrexittweets>.
- Ben Hamner. Hillary Clinton and Donald Trump Tweets, 2016. URL <https://www.kaggle.com/datasets/benhamner/clinton-trump-tweets>.
- Do-Gyeong Kim, Yuhwa Lee, Simon Washington, and Keechoo Choi. Modeling crash outcome probabilities at rural intersections: Application of hierarchical binomial logistic models. *Accident Analysis & Prevention*, 39(1):125–134, 2007.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- Daniel Loureiro, Francesco Barbieri, Leonardo Neves, Luis Espinosa Anke, and Jose Camacho-Collados. TimeLMs: Diachronic language models from twitter. In *Proceedings of ACL 2022: System Demonstrations*, 2022.
- Juan Antonio Marín Albaladejo and Marián Navarro-Beltrá. La polarización en la campaña de las elecciones generales de 2023. análisis de los posts de los partidos españoles en x. *EPSIR European Public & Social Innovation Review*, 10:1–22, 2025. doi: 10.31637/epsir-2025-474.
- Alan Mislove, Sune Lehmann, Yong-Yeol Ahn, Jukka-Pekka Onnela, and J Niels Rosenquist. Understanding the demographics of twitter users. In *Proceedings of the International AAAI Conference on Web and Social Media*, volume 5, pages 554–557, 2011.
- Saif M Mohammad, Svetlana Kiritchenko, Parinaz Sobhani, Xiaodan Zhu, and Colin Cherry. Semeval-2016 task 6: Detecting stance in tweets. In *Proceedings of the 10th International Workshop on Semantic Evaluation (SemEval-2016)*, pages 31–41, San Diego, California, 2016. Association for Computational Linguistics. <https://aclanthology.org/S16-1003/>.
-

- Arun Mohan. Understanding distil bert in depth. Medium, November 2022. <https://arunm8489.medium.com/understanding-distil-bert-in-depth-5f2ca92cf1ed>.
- Alameen Najjar, Shun'ichi Kaneko, and Yoshikazu Miyanaga. Combining satellite imagery and open data to map road safety. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 31, 2017.
- Dat Quoc Nguyen, Thanh Vu, and Anh Tuan Nguyen. Bertweet: A pre-trained language model for english tweets. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, pages 9–14. Association for Computational Linguistics, 2020. Accedido: 19/02/2026. <https://aclanthology.org/2020.emnlp-demos.2/>.
- World Health Organization. Despite notable progress, road safety remains urgent global issue, 2023. <https://www.who.int/news/item/13-12-2023-despite-notable-progress-road-safety-remains-urgent-global-issue>.
- Juan Ignacio Pérez. Cómo funcionan los Transformers en español. NLP, GPT, BERT. Aprende Machine Learning, 2020. <https://www.aprendemachinelearning.com/como-funcionan-los-transformers-espanol-nlp-gpt-bert/>.
- Juan Manuel Pérez, Damián Ariel Furman, Laura Alonso Alemany, and Franco M. Luque. Robertuito: a pre-trained language model for social media text in Spanish. In *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, pages 7235–7243, Marseille, France, 2022. European Language Resources Association. URL <https://aclanthology.org/2022.lrec-1.785>.
- Sara Rosenthal, Noura Farra, and Preslav Nakov. Semeval-2017 task 4: Sentiment analysis in twitter. In *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, pages 502–518, Vancouver, Canadá, 2017. Association for Computational Linguistics.
- Rohan Roy. 2024 US Presidential Elections Twitter Data, 2025. URL <https://www.kaggle.com/dsv/11432993>.
- Chengcheng Shao, Giovanni Luca Ciampaglia, Onur Varol, Kai-Cheng Yang, Alessandro Flammini, and Filippo Menczer. The spread of low-credibility content by social bots. *Nature Communications*, 9(1):4787, 2018. doi: 10.1038/s41467-018-06930-7.
- Theresa Wilson, Janyce Wiebe, and Paul Hoffmann. Recognizing contextual polarity in phrase-level sentiment analysis. In *Proceedings of the Conference on Human Language Technology and Empirical Methods in Natural Language Processing (HLT/EMNLP)*, pages 347–354, Vancouver, Canadá, 2005. Association for Computational Linguistics.
- Lingfei Wu, Peng Cui, Jian Pei, Liang Zhao, and Xiaojie Guo. *Graph neural networks: foundation, frontiers and applications*. 2022.
-